

The Relative Monadic Metalanguage

JACK LIELL-COCK, University of Oxford, United Kingdom

ZEVI SHIRAZI, University of Oxford, United Kingdom

SAM STATON, University of Oxford, United Kingdom

Relative monads provide a controlled view of computation. We generalise the monadic metalanguage to a relative setting and give a complete semantics with strong relative monads. Adopting this perspective, we generalise two existing program calculi from the literature. We provide a linear-non-linear language for graded monads, LNL-RMM, along with a semantic proof that it is a conservative extension of the graded monadic metalanguage. Additionally, we provide a complete semantics for the arrow calculus, showing it is a restricted relative monadic metalanguage. This motivates the introduction of ARMM, a computational lambda calculus-style language for arrows that conservatively extends the arrow calculus.

CCS Concepts: • **Theory of computation** → **Categorical semantics**.

Additional Key Words and Phrases: relative monads, monad strength, finitary monads, graded monads, arrows.

ACM Reference Format:

Jack Liell-Cock, Zev Shirazi, and Sam Staton. 2026. The Relative Monadic Metalanguage. *Proc. ACM Program. Lang.* 10, POPL, Article 60 (January 2026), 40 pages. <https://doi.org/10.1145/3776702>

1 Introduction

Over the past 15 years, *relative monads* [1] have emerged as a version of monads that provide a more controlled view of computational effects. They have applications in programming theory (e.g. [28, 31, 42]) as well as in more mathematical directions (such as [2–4, 9, 11]). Ordinary, non-relative monads have caused a dramatic transformation across programming languages over the past 35 years, both in theory (e.g. [29, 55, 56]) and in practice (e.g. [53, 66]), as they support programming with effects in a principled, functional setting. A natural question is whether relative monads can cause a similarly profound transformation. This is the starting point for this paper.

One of the driving forces behind the adoption of ordinary monads in computer science is the monadic metalanguage, a mini programming language that precisely defines the relationships between pure functional programs and programs with effects. In this paper, we provide a **relative monadic metalanguage**, a version of the monadic metalanguage that is suited to relative monads. We show that this relative monadic metalanguage encapsulates previous calculi, namely the graded monadic metalanguage (see Section 5 and [30]) and the arrow calculus (see Section 6 and [43]).

The key aspect of monads that supports their application in programming theory is the requirement of *strength*. This is a critical component of the interpretation of the monadic metalanguage. Roughly, it allows monads and side-effecting computation to occur in context. To enable a study of the relative monadic metalanguage, we also study the notion of strength in the relative case, relating notions from the literature (Section 4).

Authors' Contact Information: Jack Liell-Cock, jack.liell-cock@cs.ox.ac.uk, University of Oxford, Oxford, United Kingdom; Zev Shirazi, zev.shirazi@cs.ox.ac.uk, University of Oxford, Oxford, United Kingdom; Sam Staton, University of Oxford, Oxford, United Kingdom.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/1-ART60

<https://doi.org/10.1145/3776702>

1.1 A relative monad primer

Recall that monads consist of a type constructor T together with methods for sequencing computations in T , typically a bind and return. In the ordinary monadic metalanguage, if A is a type, then TA is a type of computation returning results in A . But TA is itself a type, just the same sort of thing as A , and so there is no problem with also writing TTA , the type of computations that return results in the type of computations.

In many situations, this is useful, but in some situations, this higher-order type TTA is hard to manipulate mathematically, and it is also conceptually difficult. The idea of relative monads is to forbid TTA by separating types into two different sorts. The sort of the type A (we call the sort $\mathcal{A}$) is different from the sort of the type TA (we call the sort $\mathcal{C}$), so that TTA is not allowed, because we cannot apply the monad type constructor to something of the monad type.

For a concrete example, we consider the probability monad D on the category of sets. For a finite set A , the set DA is the set of probability distributions with A outcomes,

$$DA = \{p : A \rightarrow [0, 1] \mid \sum_{a \in A} p(a) = 1\}.$$

This set DA is not finite, even though A is finite. Although we *can* extend D to a constructor on all sets, the resulting set DDA is not a very familiar concept from probability theory or statistics. Random measures such as the Dirichlet distribution *are* common in statistics, but those typical examples are not contained in DDA . Therefore, it is reasonable to want to focus on DA where A is a finite set. (For more discussion of this example, see Section 3.2.)

Generally, the setting for a relative monad is a category $\mathcal{A}$ (e.g. finite sets), another category $\mathcal{C}$ (sets), and a functor $J : \mathcal{A} \rightarrow \mathcal{C}$ (every finite set is, in particular, a set). Then a relative monad is also a functor $T : \mathcal{A} \rightarrow \mathcal{C}$, which enforces the discipline that one cannot apply T to itself. (See Section 2 for the full definitions.)

1.2 Relative monadic metalanguages

This paper revolves around the study of metalanguages for relative monads. We generalise ordinary monadic metalanguages to this relative setting and enforce this separation of types into two sorts in our new language. This leads to a bifurcation which generalises the monadic programming paradigm in two directions.

Paradigm 1: The first paradigm entails programs between $\mathcal{C}$ types. In this paradigm, a relative monad is a *restriction* on where the computations may occur. In particular, a computation of a computation may not exist, so they may not be arbitrarily iterated. For example, $\mathcal{A}$ types may be finite types, while the effectful programs may produce infinite results of type $\mathcal{C}$. The finite distributions monad, which encodes probabilistic choice, is one such example. While the restriction to $\mathcal{A}$ may seem inhibitive, it greatly simplifies program reasoning and analysis.

Paradigm 2: The second paradigm is concerned with programs between $\mathcal{A}$ types. For $\mathcal{C}$ types to have any utility, there must be a way to move from them back to the $\mathcal{A}$ universe. Formally, this is usually instantiated as an adjunction, for example, in Linear Logic [7], Call by Push Value [39], and the Enriched Effect Calculus [15]. This paradigm permits computations that are beyond the expressiveness of the $\mathcal{A}$ types to be performed in $\mathcal{C}$, before being transported back to programs in $\mathcal{A}$. This allows a more general notion of computation since $\mathcal{C}$ types are not necessarily *first-class*, i.e. the types of inputs and outputs to programs, but can model more general behaviour than those in $\mathcal{A}$.

In programming language theory, and in the monadic metalanguage, we are mainly concerned with *strong monads* [35, 36, 49]. When sequencing effectful programs together, non-strong monads are not sufficient because they do not allow the threading of contexts into later programs. Strong

monads provide a method for pairing a *pure* context with a computational result to be passed into the next program.

To interpret a relative monadic metalanguage, it thus makes sense to define the notion of *strong relative monads* [60, 65]. However, there are subtleties to account for when making this generalisation. Many equivalent definitions of strong monads [49] are no longer equivalent when transported to the relative setting. Varying definitions of strong relative monads have been used to generalise and express a range of programming concepts [59, 60, 65]. To get a holistic view of strength for relative monads, we collate the definitions and outline the relationships between them. We provide a hierarchy of subsumptions and specify the criteria for when the subsumptions flatten into equivalences. In particular, we re-establish the connection between strong monads and enriched monads in the relative setting (Theorem 4.6). These results feed into our understanding of the relative monadic metalanguage and are crucial for providing semantic proofs about its applications.

1.3 Applications of the Relative Monadic Metalanguage

Using the theory we develop, we extend the relative monadic metalanguage to capture two concepts: graded monads and arrows. Both of these notions have been modelled by program calculi previously, but using the power of strong relative monads, we provide semantically backed program calculi for these concepts. Both lead to useful conservative extensions of the calculi in the previous literature.

1.3.1 Graded monads. Graded monads [18, 22, 30, 37, 51, 64] are a generalisation of monads that give a semantic version of *effect usage analysis*. They have been used to give semantic accounts of memory access analysis with the state monad, output analysis for the writer monad, and branch cuts in non-determinism (e.g. [30, 31]). Moreover, they have been used in probabilistic programs to track non-deterministic choice [42], and to track execution traces to permit programmable inference [41]. A big advantage of graded monads is that they can implement programs that are *cost-sensitive*. A graded monadic metalanguage can be used for cost analysis of programs [12] by keeping track of the resources in the grades. Graded monads have been established as an instance of enriched relative monads [48], and in Theorem 5.5 we extend this connection in the strong case. The correspondence enables our development of a relative monadic style language for graded monads (which we call LNL-RMM). This language follows paradigm 2, where it is built on top of the linear-non-linear calculus [7]. This calculus has two sorts of types, linear and non-linear, and constructs that allow transitions between them. Linear type theory is a *resource-sensitive* type theory that doesn't allow discarding or duplication of variables in contexts. It is, therefore, suitable for reasoning directly with grades, since we often interpret grades as tracking of effect usage. In particular, for cost analysis of programs [12], the linearity here is used to ensure we cannot do any computation for free. We therefore explicitly have additional *grade types* in the linear sort that allow us to directly program with grades. By encoding a strong relative monad over the linear to non-linear transition, we obtain a language for graded monads. In previous languages, gradings are explicit parameters, whereas ours are types within the language, and the graded monad is framed as a function from grades to a relative monad (5). These first-class grades enable more flexible programming and code reuse, as described in Section 5.

1.3.2 Arrows. Arrows are a programming language paradigm that generalise monads [25]. While monads are a tool for sequencing effectful computations, arrows model computation that is not strictly sequential. They have a broad range of applications in functional programs, such as for parsers and printers [27], web interaction [25], circuits [52], graphical user interfaces [13], and robotics [24]. The established connection between arrows and strong relative monads [1, 65] motivates our development of a relative monadic language for arrows (which we call ARMM).

This language also follows paradigm 2, where the standard programs are between $\mathcal{A}$ types, while C types model parametric computations on inputs of type $\mathcal{A}$. The language we develop is a conservative extension of the arrow calculus [43] (Theorem 6.6). It has all the expressiveness of arrows but additionally allows for variables of arrow computations, which promotes the reusability and flexibility of the programs. Moreover, it has more principled semantics, making it amenable to generalisations with additional type formers, other variants such as a linear language for arrows, or semantic proofs of language properties.

1.4 Contributions

Our main contributions can be summarised as follows:

- We introduce the relative monadic metalanguage (which we call RMM) in Section 3.1 as a framework for languages in the paper and provide a semantics via strong relative monads.
- We introduce the language LNL-RMM as a more expressive language for reasoning with graded monads and provide a semantic proof in Theorem 5.13 that it is a conservative extension of the graded monadic metalanguage.
- We introduce the language ARMM as a more expressive language for programming with arrows and provide a semantic proof in Theorem 6.6 that it is a conservative extension of the arrow calculus.

Our framework calculus RMM and strong relative monads are introduced in Section 3, including an example calculus for reasoning with finitary monads. Section 4 proves a correspondence between strong relative monads and enriched relative monads and relates other notions of strong relative monads in the literature. Section 5 introduces the LNL-RMM and proves conservativity by showing a correspondence between strong graded monads and strong relative monads in Theorem 5.5. Section 6 introduces ARMM and proves conservativity by providing a complete semantics for the arrow calculus in Theorem 6.2.

2 Monads, relative monads and a simple calculus

In this section, we review the definition of monads and relative monads and introduce a simple calculus (with unary contexts) for relative monads [50]. We present the definition of a monad in its *Kleisli triple form* which aligns with their implementation in programming languages.

Definition 2.1. A monad $(T, \eta, (-)^*)$ on a category C consists of:

- An object $TX \in C$ for each $X \in |C|$,
- A morphism $\eta_X : X \rightarrow TX$ for each $X \in |C|$ called the *unit*
- A collection of maps $(-)^* : C(X, TY) \rightarrow C(TX, TY)$ called the *Kleisli extension* or *bind*

such that for $f : X \rightarrow TY$ and $g : Y \rightarrow TZ$, the following equations hold:

$$(\eta_X)^* = \text{id}_{TX} \qquad f^* \circ \eta_X = f \qquad g^* \circ f^* = (g^* \circ f)^*$$

From this data, we can define a unique functor structure on T such that $(-)^*$ and η are natural in $X, Y \in |C|$, given by $Tf = (\eta_Y \circ f)^*$ for $f : X \rightarrow Y$ [47]. Monads are often presented in their *monoid form* (T, η, μ) [44], where here $\mu_X = (\text{id}_{TX})^*$. From the monoid form, we can recover Definition 2.1 by defining $f^* = \mu_Y \circ Tf$ for $f : X \rightarrow TY$ [46].

Definition 2.2. Let $|\mathcal{A}|$ be a collection of objects, C a category, and $J : |\mathcal{A}| \rightarrow |C|$ a mapping of objects, called the *root*. A J -relative monad $(T, \eta, (-)^*)$ consists of

- An object $TA \in |C|$ for each $A \in |\mathcal{A}|$
- A morphism $\eta_A : JA \rightarrow TA$ in C for each $A \in |\mathcal{A}|$
- A collection of maps $(-)^* : C(JA, TB) \rightarrow C(TA, TB)$

all such that for $f : JA \rightarrow TB$ and $g : JB \rightarrow TC$, the following equations hold:

$$(\eta_A)^* = \text{id}_{TA} \quad f^* \circ \eta_A = f \quad g^* \circ f^* = (g^* \circ f)^*$$

A *morphism of J -relative monads* $\gamma : (T, \eta, (-)^*) \rightarrow (S, \nu, (-)^\dagger)$ consists of maps $\gamma_A : TA \rightarrow SA$ satisfying, for any $f : JA \rightarrow TB$,

$$\nu_A = \gamma_A \circ \eta_A, \quad \gamma_B \circ f^* = (\gamma_B \circ f)^\dagger \circ \gamma_A.$$

We denote the category of J -relative monads and morphisms as $\text{Mon}(J)$.

When $\mathcal{A}$ is a category with objects $|\mathcal{A}|$ and $J : |\mathcal{A}| \rightarrow |C|$ extends to a functor $J : \mathcal{A} \rightarrow C$, the mapping on objects $T : |\mathcal{A}| \rightarrow |C|$ has a unique functor structure such that $(-)^*$ and η are natural in $A, B \in |\mathcal{A}|$, defined by $Tf = (\eta_A \circ Jf)^*$. This additionally makes the morphisms of J -relative monads, $\gamma_A : TA \rightarrow SA$, natural in A .

Remark. Relative monads are often defined with respect to roots that are *functors* [1, 2], rather than mappings on objects. Since any mapping on objects $J : |\mathcal{A}| \rightarrow |C|$ can be seen as a functor from the discrete category on $|\mathcal{A}|$, these definitions are equivalent. In Section 4.1, we will consider the enriched setting, where discrete categories may not exist. Hence, this equivalence breaks, and there are benefits to considering mappings on objects.

Example 2.3. $\text{Mon}(\text{id}_C)$ is the category of monads on C and monad morphisms (see [6]).

Example 2.4. If $(T, \eta, (-)^*)$ is a monad on C , then we can define a relative monad $(TJ, \eta^J, (-)^{*J})$ by restricting the monad to objects under the image of J , where for $f : JA \rightarrow TJB$:

$$\eta_A^J = \eta_{JA} : JA \rightarrow TJA \quad f^{*J} = f^* : TJA \rightarrow TJB$$

This assignment on objects extends to a functor $\text{res}_J : \text{Mon}(\text{id}_C) \rightarrow \text{Mon}(J)$. We say T *restricts* to a J -relative monad $\tilde{T}$ if $\tilde{T} \cong \text{res}_J T$.

Relative monads are often studied for roots that are *well-behaved* [1]. In these cases, the root satisfies additional conditions so that any relative monad is the restriction of an ordinary monad. When programming with effects presented by algebraic operations (see [55, 56]), we can take $\mathcal{A}$ to be a category of *arities*, so extensions of a relative monad T for a well-behaved $J : \mathcal{A} \rightarrow C$ can be viewed as a monad with arities in $\mathcal{A}$ [9, 11].

Example 2.5. The inclusion $J : \text{FinSet} \rightarrow \text{Set}$ is well-behaved, and every J -relative monad is the restriction of an ordinary monad on Set . The functor res_J defines an equivalence between J -relative monads and *finitary* monads on Set .

Example 2.6. The Yoneda embedding $\gamma : C \rightarrow \widehat{C}$ (where $\widehat{C} = [C^{\text{op}}, \text{Set}]$) is also well-behaved, and γ -relative monads correspond to *promonads* [10] or *weak arrows* [1]. These have been used to provide semantics for computational effects that cannot be captured by monadic style programming [25, 57].

2.1 A unary context metalanguage for relative monads (URMM)

This section introduces a limited calculus for relative monads that only permits unary contexts. While this limits the applicability of the calculus, it provides a template for the more complex versions introduced later. In particular, in Section 3, we extend this calculus to admit product types and multiple-arity contexts. Fixing a category $\mathcal{A}$, the language $\text{URMM}(\mathcal{A})$ has the following types.

$$\text{Types: } X, Y ::= JA \mid TA \quad (A \in |\mathcal{A}|)$$

A judgement is of the form $x : X \vdash u : Y$, and terms are built inductively.

$$\frac{}{x : X \vdash x : X} \quad \frac{x : X \vdash u : JA}{x : X \vdash fu : JB} (f \in \mathcal{A}(A, B))$$

$$\frac{x : X \vdash u : JA}{x : X \vdash \text{return } u : TA} \quad \frac{x : X \vdash u : TA \quad y : JA \vdash t : TB}{x : X \vdash \text{do } y \leftarrow u \text{ in } t : TB}$$

In addition to imposing the standard reflexivity, symmetry, transitivity, substitution, and congruence laws for equality of terms (see Appendix B), we have equations:

$$\frac{x : X \vdash u : JA}{x : X \vdash \text{id}_A u = u : JA} \quad \frac{x : X \vdash u : JB}{x : X \vdash (g \circ f) u = g(f u) : JC} (f \in \mathcal{A}(A, B), g \in \mathcal{A}(B, C))$$

$$\frac{x : X \vdash u : JA \quad y : JA \vdash t : TB}{x : X \vdash \text{do } y \leftarrow \text{return } u \text{ in } t = t[u/y] : TB} \quad \frac{x : X \vdash u : TA}{x : X \vdash \text{do } y \leftarrow u \text{ in return } y = u : TA}$$

$$\frac{x : X \vdash u : TA \quad y : JA \vdash t : TB \quad z : JB \vdash v : TC}{x : X \vdash \text{do } z \leftarrow (\text{do } y \leftarrow u \text{ in } t) \text{ in } v = \text{do } y \leftarrow u \text{ in } (\text{do } z \leftarrow t \text{ in } v) : TC}$$

In categorical semantics, it is usual to interpret types as objects in a category, and terms as morphisms. The additional structure we impose here is interpreted as a relative monad.

Definition 2.7. A model of $\text{URMM}(\mathcal{A})$ consists of:

- (1) A category C
- (2) A functor $J : \mathcal{A} \rightarrow C$
- (3) A J -relative monad $(T, \eta, (-)^*)$

Given a model of $\text{URMM}(\mathcal{A})$, (C, J, T) , we define the interpretation of a judgement $\llbracket x : X \vdash u : Y \rrbracket$ to be a morphism $\llbracket X \rrbracket \rightarrow \llbracket Y \rrbracket$ in C (where $\llbracket X \rrbracket$ is defined in the evident way). We say an equation $x : X \vdash u = v : Y$ is valid in (C, J, T) if $\llbracket x : X \vdash u : Y \rrbracket = \llbracket x : X \vdash v : Y \rrbracket$. We inductively define the interpretation of terms as follows:

$$\begin{aligned} \llbracket x : X \vdash x : X \rrbracket &= \text{id}_{\llbracket X \rrbracket} \\ \llbracket x : X \vdash fu : JC \rrbracket &= Jf \circ \llbracket x : X \vdash u : JB \rrbracket \\ \llbracket x : X \vdash \text{return } u : TB \rrbracket &= \eta_B \circ \llbracket x : X \vdash u : JB \rrbracket \\ \llbracket x : X \vdash \text{do } y \leftarrow u \text{ in } t : TC \rrbracket &= \llbracket y : JB \vdash t : TC \rrbracket^* \circ \llbracket x : X \vdash u : JB \rrbracket \end{aligned}$$

PROPOSITION 2.8. *The models of $\text{URMM}(\mathcal{A})$ are a sound and complete semantics.*

PROOF. The proof of soundness is done by structural induction and is standard. Completeness follows from the construction of a term model $(\mathcal{F}, J, T)$. $\mathcal{F}$ has objects JA and TA for $A \in |\mathcal{A}|$ and morphism $X \rightarrow Y$ are equivalence classes of terms $[x : X \vdash u : Y]$ under the equational theory. The functorial action of J is defined by $[x : JA \vdash fu : JB]$ while the return and do terms provide the data for the relative monad T . $\square$

3 Strong monads, strong relative monads and the relative monadic metalanguage

Program contexts are often larger than a single variable. If we want to sum two numbers together, then we already require more than one variable. However, monads do not have sufficient structure

for interpreting the sequencing of computations with multiple free variables. For example, say we intend to sequence the following two effectful programs.

$$\Gamma \vdash u : TA \qquad \Gamma, x : A \vdash t : TB$$

Interpreted as morphisms, these become

$$\llbracket u \rrbracket : \llbracket \Gamma \rrbracket \rightarrow T\llbracket A \rrbracket, \qquad \llbracket t \rrbracket : \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \rightarrow T\llbracket B \rrbracket.$$

To sequence these, we need to produce from $\llbracket t \rrbracket$ a map $\llbracket \Gamma \rrbracket \times T\llbracket A \rrbracket \rightarrow T\llbracket B \rrbracket$, i.e. apply Kleisli extension *under the context* Γ . Strong monads provide the required semantics for such an operation. In the general setting, strong monads are defined with respect to monoidal products, of which the Cartesian product is an instance. We employ this setting for a linear type system (see [7, 8, 19]) in Section 5, so we introduce the concepts at this generality.

Definition 3.1 ([44]). A *monoidal category* $(C, \otimes, I, \lambda, \rho, \alpha)$ consists of a category C , an object $I \in |C|$, and a functor $\otimes : C \times C \rightarrow C$, equipped with natural isomorphisms,

$$\lambda_X : I \otimes X \rightarrow X, \quad \rho_X : X \otimes I \rightarrow X, \quad \alpha_{X,Y,Z} : (X \otimes Y) \otimes Z \rightarrow X \otimes (Y \otimes Z),$$

called the *left unitor*, *right unitor* and *associator*, satisfying coherence equations (see [44]). A monoidal category is *symmetric* when there is an additional natural isomorphism $\sigma_{X,Y} : X \otimes Y \rightarrow Y \otimes X$, called the *symmetry*, satisfying $\sigma_{Y,X} \circ \sigma_{X,Y} = \text{id}_{X \otimes Y}$ and additional coherences (see [44]).

A strong monad has the same data as a monad, but it can extend variables under the tensor.

Definition 3.2 ([49, 50]). A *strong monad* $(T, \eta, (-)^*)$ on a symmetric monoidal category $(C, \otimes, I)$ consists of

- An object $TX \in C$ for each $X \in |C|$
- A morphism $\eta_X : X \rightarrow TX$ for each $X \in |C|$
- A collection of maps $(-)^* : C(\Gamma \otimes X, TY) \rightarrow C(\Gamma \otimes TX, TY)$ natural in $\Gamma \in |C|$

such that, for $f : \Gamma \otimes X \rightarrow TY$ and $g : \Delta \otimes Y \rightarrow TZ$, the following three equations hold:

$$(\eta_X \circ \lambda_X)^* = \lambda_{TX} \quad f^* \circ (\Gamma \otimes \eta_X) = f \quad g^* \circ (\Delta \otimes f^*) \circ \alpha_{\Delta, \Gamma, TX} = (g^* \circ (\Delta \otimes f) \circ \alpha_{\Delta, \Gamma, X})^*$$

Strong monads were originally presented using their *monoid form* [35, 36]. We discuss these equivalent definitions further in Section 4. As in Definition 2.2, the *Kleisli triple form* of a strong monad is more amenable to generalisation to the relative setting.

Definition 3.3. Let $|\mathcal{A}|$ be a collection of objects, $(C, \otimes, I)$ a symmetric monoidal category, and $J : |\mathcal{A}| \rightarrow |C|$ a mapping on objects. A *strong J-relative monad* $(T, \eta, (-)^*)$ on C consists of:

- An object $TA \in |C|$ for each $A \in |\mathcal{A}|$,
- A morphism $\eta_A : JA \rightarrow TA$ for each $A \in |\mathcal{A}|$,
- A collection of maps $(-)^* : C(\Gamma \otimes JA, TB) \rightarrow C(\Gamma \otimes TA, TB)$ natural in $\Gamma \in C$.

all such that for $f : \Gamma \otimes JA \rightarrow TB$ and $g : \Delta \otimes JB \rightarrow TC$, the following equations hold:

$$(\eta_A \circ \lambda_{JA})^* = \lambda_{TA} \quad f^* \circ (\Gamma \otimes \eta_A) = f \quad g^* \circ (\Delta \otimes f^*) \circ \alpha_{\Delta, \Gamma, TA} = (g^* \circ (\Delta \otimes f) \circ \alpha_{\Delta, \Gamma, JA})^*$$

A *morphism of strong J-relative monads* $\gamma : (T, \eta, (-)^*) \rightarrow (S, \nu, (-)^\dagger)$ consists of, for each $A \in |\mathcal{A}|$, maps $\gamma_A : TA \rightarrow SA$ satisfying, for any $f : \Gamma \otimes JA \rightarrow TB$:

$$\nu_A = \gamma_A \circ \eta_A \qquad \gamma_B \circ f^* = (\gamma_B \circ f)^\dagger \circ (\Gamma \otimes \gamma_A)$$

We write $\mathbf{SMon}(J)$ for the category of strong J -relative monads.

Remark. Other definitions of strong relative monad have been suggested in the literature [2, 65]. We explore the connections between these definitions further in Section 4. For non-symmetric monoidal categories, the number of inequivalent definitions fragments further. The most applicable notion for (non-commutative) linear type theories is that of *bistrong* J -relative monads, which we describe in Appendix A. This notion is closely related to *strong multicategorical relative monads* [60].

Given a strong J -relative monad $(T, \eta, (-)^*)$, it has an *underlying* J -relative monad $(T, \eta, (-)^{*U})$, given by $f^{*U} = (f \circ \lambda_{JA})^* \circ \lambda_{TA}^{-1}$ for $f \in C(JA, TB)$. This extends T to a functor when J is a functor as in the non-strong case, and defines the object mapping for a forgetful functor $\mathbf{SMon}(J) \rightarrow \mathbf{Mon}(J)$. The restriction functor $\text{res}_J : \mathbf{Mon}(\text{id}_C) \rightarrow \mathbf{Mon}(J)$ extends via this forgetful functor to a restriction $\text{sres}_J : \mathbf{SMon}(\text{id}_C) \rightarrow \mathbf{SMon}(J)$.

$$\begin{array}{ccc} \mathbf{SMon}(\text{id}_C) & \xrightarrow{\text{sres}_J} & \mathbf{SMon}(J) \\ \downarrow & & \downarrow \\ \mathbf{Mon}(\text{id}_C) & \xrightarrow{\text{res}_J} & \mathbf{Mon}(J) \end{array}$$

Example 3.4. When $C = \mathbf{Set}$, every J -relative monad $(T, \eta, (-)^*)$ is uniquely strong, where the extension maps $(-)^{\dagger} : \mathbf{Set}(X \times JA, TB) \rightarrow \mathbf{Set}(X \times TA, TB)$ are given by $f^{\dagger} = \lambda(x, u). (f(x, -))^*(u)$.

Example 3.5. $\mathbf{SMon}(\text{id}_C)$ is the category of strong monads and strong monad morphisms [49].

3.1 The relative monadic metalanguage (RMM)

Using strong relative monads, we now extend the unary context language URMM in Section 2.1 to a language RMM that admits product types and sequences computations under arbitrary contexts. It is analogous to the monadic metalanguage [50], but restricts the type of sequencing allowed. It is also reminiscent of a fragment of Call-by-Push-Value [39], but the typical relative monads are not full Call-by-Push-Value models (see also [28]). In Section 5 and Section 6, we use this relative monadic metalanguage as a foundation for modelling generalisations of monadic effects.

Fixing a category $\mathcal{A}$, the language $\text{RMM}(\mathcal{A})$ is defined as follows.

$$\begin{array}{ll} \text{Types:} & X, Y ::= 1 \mid X \times Y \mid JA \mid TA \quad (A \in |\mathcal{A}|) \\ \text{Contexts:} & \Gamma ::= \emptyset \mid \Gamma, x : X \end{array}$$

Terms are generated inductively. The full equational theory is given in Appendix B.1.

$$\begin{array}{c} \frac{}{\Gamma, x : X \vdash x : X} \quad \frac{}{\Gamma \vdash () : 1} \quad \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash \pi_1 u : X} \quad \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash \pi_2 u : Y} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash (u, t) : X \times Y} \\ \\ \frac{\Gamma \vdash u : JA}{\Gamma \vdash f u : JB} (f \in \mathcal{A}(A, B)) \quad \frac{\Gamma \vdash u : JA}{\Gamma \vdash \text{return } u : TA} \quad \frac{\Gamma \vdash u : TA \quad \Gamma, x : JA \vdash t : TB}{\Gamma \vdash \text{do } x \leftarrow u \text{ in } t : TB} \end{array}$$

As in Section 2.1, we interpret terms and types in a category, now with a *strong* relative monad.

Definition 3.6. A *model* of $\text{RMM}(\mathcal{A})$ consists of:

- (1) A Cartesian category C
- (2) A functor $J : \mathcal{A} \rightarrow C$
- (3) A strong J -relative monad $(T, \eta, (-)^*)$

If $\Gamma = a_1 : A_1, \dots, a_n : A_n$, then its interpretation is given by the product $\prod_{i=1}^n \llbracket A_i \rrbracket$ in C . A term $\Gamma \vdash u : X$ is interpreted as a morphism $\llbracket \Gamma \rrbracket \rightarrow \llbracket X \rrbracket$ in C . The only significant difference from the

interpretation of terms in $\text{URMM}(\mathcal{A})$ is the sequencing terms.

$$\llbracket \Gamma \vdash \text{do } x \leftarrow u \text{ in } t : TB \rrbracket = (\llbracket \Gamma, x : JA \vdash t : TB \rrbracket)^* \circ (1_\Gamma, \llbracket \Gamma \vdash u : TA \rrbracket)$$

THEOREM 3.7. *The models of $\text{RMM}(\mathcal{A})$ are a sound and complete semantics.*

PROOF. As in Proposition 2.8, the proof of completeness is by constructing the term model (defined analogously to the term model in Proposition 2.8). The proof of soundness is also by structural induction, and by first verifying a substitution lemma $\llbracket \Gamma \vdash t[u/x] : Y \rrbracket = \llbracket \Gamma, x : X \vdash t : Y \rrbracket \circ (1_{\llbracket \Gamma \rrbracket}, \llbracket \Gamma \vdash u : X \rrbracket)$. Note that the equations in Definition 3.3 for a strong relative monad reduce to the equations in Appendix B.1 on a Cartesian category. $\square$

3.2 Strongly finitary monads in the relative monadic metalanguage

The relative monadic metalanguage can be thought of as a very simple first-order programming language. The strength allows us to reason in context, which is crucial for a programming language. The relative aspect allows us to keep to a first order. Even if the ordinary monadic metalanguage does not have explicit function types, it is not really first-order, because it can talk about computations that return computations (*TTA*). This type would be represented in Standard ML with the type `unit -> (unit -> A)`, since functions of unit type are thunks and `unit -> (-)` is a monad; notice that it is indeed higher order there.

This cut-down, first-order setting is ideal for equationally presenting notions of computational effect, without worrying about aspects of higher-order computation that are often orthogonal. The idea is that a computational effect can be presented by giving a first-order programming language together with equations that it satisfies. One can often add higher-order features separately, afterwards.

In this section, we demonstrate how this cut-down approach is already being taken in the literature, and can be thought of as making essential use of an informal flavour of the relative monadic metalanguage, with examples from both probability and finitary **Set**-monads (Section 3.2.1) and local store and more general finitary enriched monads (Section 3.2.2).

3.2.1 Finitary monads, and examples from probabilistic programming. Recall that a *finitary monad* can be given by a relative monad for the embedding $J : \mathbf{FinSet} \rightarrow \mathbf{Set}$, of finite sets into sets (Example 2.5). These are necessarily strong (Example 3.4). We can regard the objects of **FinSet** as being natural numbers.

For illustration, recall from the introduction the finitary monad of probability distributions. The relative monad $D : \mathbf{FinSet} \rightarrow \mathbf{Set}$ has Dn the probabilities on n outcomes,

$$Dn = \{(p_1 \dots p_n) \mid p_i \geq 0 \text{ \& \; } \sum_{i=1}^n p_i = 1\}.$$

Often, a finitary monad is presented by an algebraic theory with operations and equations. In the case of D , this can be presented by Stone's barycentric algebras [63]. This includes a binary operation $x \oplus y$, meaning 'toss a fair coin and choose x or y ', and axioms

$$x \oplus x = x, \quad (u \oplus x) \oplus (y \oplus z) = (u \oplus y) \oplus (x \oplus z), \quad x \oplus y = y \oplus x. \quad (1)$$

If we restrict the p_i s to dyadic rationals, i.e. $p_i = \frac{a}{2^n}$, this is a full presentation of D . This gives part of a correspondence between finitary monads and algebraic theories.

On the other hand, the relative monadic metalanguage for D supports base types from $\mathcal{A} = \mathbf{FinSet}$, and a term

$$\vdash \text{coin} : T2$$

which is intuitively a coin toss operation, returning a boolean value (2), and is interpreted as the probability distribution (0.5, 0.5). This is a finite probabilistic programming language, similar to DICE [23], which allows sequencing probabilistic operations and calculating results.

In this setting, Stone's three equations (1) become

$$\begin{aligned} \text{do } x \leftarrow \text{coin in return } () &= \text{return } () \\ \text{do } x \leftarrow \text{coin in do } y \leftarrow \text{coin in return } (x, y) &= \text{do } y \leftarrow \text{coin in do } x \leftarrow \text{coin in return } (x, y) \quad (2) \\ \text{do } x \leftarrow \text{coin in return } (\neg x) &= \text{coin} \end{aligned}$$

where $\neg : 2 \rightarrow 2$ is the boolean negation.

This program-based way of specifying the equations is complete in the following sense. For any pair of programs $J\Gamma \vdash t, u : Tn$, the equation $t = u$ is derivable from the laws of $\text{RMM}(\mathcal{A})$ if and only if it is true in the model.

This means that we can use program equations to directly specify a finitary monad, instead of algebraic theories, as long as we restrict to programs in the relative monadic metalanguage.

As an aside, we note that the relative setting here is useful specifically in probabilistic programming for at least three reasons.

- (1) Finite distributions extend to a monad $\tilde{D}$ on Set , the monad of finitely supported probability distributions. However, $\tilde{D}\tilde{D}n$ does not contain the famous distributions-on-distributions from statistics, such as the Dirichlet distribution, because they are not finitely supported. There are other probability monads on other categories that do support the Dirichlet distribution [20], but for this particular D it is statistically strange to consider $\tilde{D}\tilde{D}n$.
- (2) The inference method in the DICE language works on fixed finite spaces. Thus, it is crucial to work with Dn for some given finite n , rather than DS for some potentially infinite set.
- (3) The synthetic approach to probability theory based on Markov categories can be formulated in terms of relative monads [16, §10.19]. The restriction seems to be crucial for maintaining information flow axioms, which typically fail in models of a full monadic lambda calculus [17].

3.2.2 Kelly-Power style presentations, distributive Freyd categories, nominal sets, and local store. We have noted that the dyadic distributions monad could be presented by three program equations (2). In general, if monads are to be used for computational purposes, this is a helpful way to specify them. This is a point made by [55, p. 13] and [61], where it is shown that, for instance, a local store monad (with dynamically allocated references) can be presented by program equations such as

$$x := y; !x \quad = \quad x := y; \text{return } y \quad (3)$$

$$\text{do } x \leftarrow \text{ref}(u) \text{ in } x := v; \text{return } x \quad = \quad \text{do } x \leftarrow \text{ref}(v) \text{ in return } x \quad (4)$$

We now explain how this kind of presentation essentially amounts to formal equational reasoning in the relative monadic metalanguage. A subtle point is that the dynamically allocated references do not belong to an ordinary finite set, and that leads us beyond finitary Set -monads. We revisit this example more precisely at the end of this subsection.

In fact, this general approach to presenting monads was essentially already taken by Kelly and Power [34, §5] in their formalization of algebraic presentations of strong monads, although they did not write in terms of programs. We now recall some key aspects of that framework.

Kelly and Power begin (§4) with a symmetric monoidal closed category C that is locally finitely presentable as a closed category. This has a full subcategory $\mathcal{A} = C_f$ of finitely presentable objects. In general, a finitary enriched monad on C is, in our terminology, a strong monad relative to the embedding $J : \mathcal{A} \hookrightarrow C$. A small worry is that $\mathcal{A}$ is a large category, and so they pick a set N of representatives of each isomorphism class. These might be the basic types in our language. In the case of finitary monads on Set , $\mathcal{A} = \text{FinSet}$, and N can be the set of natural numbers.

To present the theory, Kelly and Power begin with a signature, which is an assignment $B : N \rightarrow C$. This is picking out for each type $A \in N$, a C object of basic computations of type A . This is not a functor. For probability, we would put $B2 = \{\text{coin}\}$, and $Bn = \emptyset$ for $n \neq 2$. But in general, N might not range over natural numbers.

Kelly and Power then generate a relative monad FB from B . This is, in effect, terms built from an RMM calculus specialized to C , starting with basic operations $x : Jm \vdash t : FBn$ for each n .

Kelly and Power then allow a system of equations, which are essentially pairs from FB . They show that these generate an enriched finitary monad on C , and that moreover, every enriched finitary monad arises in this way.

The approach in [34] is focused on locally finitely presentable categories, but the procedure also goes through with other sound limit doctrines (e.g. [62]), giving complete approaches to presenting Freyd categories and distributive Freyd categories. These are theories of first-order programming languages with side effects, and $\text{RMM}(\mathcal{A})$ for distributive Freyd categories is precisely the fragment of FGCBV [40] without function types.

For a precise example, we consider the presentation of a monad for local store in [61]. There, the category C comprises Pitts' nominal sets [54], and $\mathcal{A}$ comprises the full subcategory of finitary strong nominal sets [54, Thm. 2.7]. We do not need the full definitions, but the important point is that C is like a category of sets but with a special object $\mathbb{A}$ of atoms, and $\mathcal{A}$ is a subcategory spanned by interpretations of first-order types. Then [61] presents local store as a strong monad relative to the inclusion $\mathcal{A} \rightarrow C$. These will be used as names of reference cells, and the equivariance of nominal sets maintains that they are abstract – there is no pointer arithmetic. We also assume a strong nominal set $\mathbb{V}$ of storable values.

The theory is generated by basic programs,

$$x : J\mathbb{A}, y : J\mathbb{V} \vdash x := y : T1 \quad x : J\mathbb{A} \vdash !x : T\mathbb{A} \quad y : J\mathbb{V} \vdash \text{ref}(y) : T\mathbb{A}$$

for assign, lookup, and allocating a new reference cell, with equations such as (3) and (4). The paper discusses completeness of the system of 14 equations and shows that they present a monad, reasoning using a language that is essentially an instantiation of RMM. In equation (3), the strength allows us to use the variable x twice on the left-hand side, and y twice on the right.

4 Comparing notions of strong relative monads

Strong monads have several equivalent presentations [49]; however, these equivalences are not all maintained when they are transported to the relative setting. In this section, we recall notions of strong monads and describe their corresponding generalisations. We show when equivalent notions are preserved, and provide criteria for recovering them when they are not. Understanding these correspondences is crucial in the development of refined semantic type theories extending RMM. In particular, our results about graded monads in Section 5 rely on connections with *enriched relative monads* [2, 48]. Additionally, our semantics of the arrow calculus in Section 6 uses a weaker notion of strength.

We begin by recalling the categorical background for these correspondences.

Definition 4.1 ([33]). Let $(\mathcal{V}, \otimes, I)$ be a symmetric monoidal category. A $\mathcal{V}$ -enriched category C consists of:

- A collection of objects $|C|$
- A $\mathcal{V}$ -object of morphisms $C[X, Y]$ for each $X, Y \in |C|$
- An identity morphism $\text{id}_X : I \rightarrow C[X, X]$ for each $X \in |C|$
- Composition morphisms $\bullet_{X,Y,Z} : C[Y, Z] \otimes C[X, Y] \rightarrow C[X, Z]$ for $X, Y, Z \in |C|$

such that composition and identities satisfy associativity and unitality equations (see [33]).

A $\mathcal{V}$ -enriched functor $J : \mathcal{A} \rightarrow \mathcal{C}$ between $\mathcal{V}$ -enriched categories $\mathcal{A}$ and $\mathcal{C}$ consists of a mapping on objects $J : |\mathcal{A}| \rightarrow |\mathcal{C}|$ along with morphisms $J_{A,B} : \mathcal{A}[A, B] \rightarrow \mathcal{C}[JA, JB]$ in $\mathcal{V}$ preserving composition and identities.

Definition 4.2 (Day monoidal structure [14]). Let $(\mathcal{C}, \otimes, I)$ be a small symmetric monoidal category. Then $(\widehat{\mathcal{C}}, \widehat{\otimes}, \widehat{I})$ is a symmetric monoidal closed category (i.e. $- \widehat{\otimes} F \dashv F \multimap -$ for all $F \in |\widehat{\mathcal{C}}|$), and in particular, is self-enriched. Here, $\widehat{I} = \gamma I_{\mathcal{C}}$ and

$$(F \widehat{\otimes} G)(X) = \int^{Y, Z \in \mathcal{C}^{\text{op}}} \mathcal{C}(X, Y \otimes_{\mathcal{C}} Z) \times FY \times GZ.$$

Definition 4.3 (Fully faithful enrichment). If $J : |\mathcal{A}| \rightarrow |\mathcal{C}|$ is a mapping on objects and $\mathcal{C}$ is a $\mathcal{V}$ -enriched category, the J -fully faithful enrichment of $\mathcal{A}$ and J is defined by $\mathcal{A}[A, B] = \mathcal{C}[JA, JB]$ and $J_{A,B} = \text{id}_{\mathcal{C}[JA, JB]}$.

These definitions allow us to present the following three notions, which give varying perspectives on strong monads. The first provides a natural semantics for monadic programming languages by allowing extension under any computational context. The second is a categorical viewpoint; it shows that a strong monad is analogous to the data for a monad in monoid form, but with additional coherent structure. The final perspective bridges the gap between these two; it says that strength is an *internalisation* of the extension operation of a monad. This is necessary for programming languages, where the terms of the language need to *internalise* the usage of bind to program with it.

PROPOSITION 4.4 ([49]). Consider a small symmetric monoidal category $\mathcal{C}$. The following data are in bijective correspondence:

- (1) A strong monad on $\mathcal{C}$, as in Definition 3.2.
- (2) A monad on $\mathcal{C}$ with strength maps $\theta_{X,Y} : X \otimes TY \rightarrow T(X \otimes Y)$ satisfying compatibility conditions with the monoidal and monad structures (see [36], also Definition 4.15 with $J = \text{id}_{\mathcal{C}}$).
- (3) A $(\widehat{\mathcal{C}}, \widehat{\otimes}, \widehat{I})$ -enriched monad (see [49], also Definition 4.5 with $J = \text{id}_{\mathcal{C}}$) on $\mathcal{C}$ with the γ -fully faithful enrichment.

In Section 4.1 and Section 4.2, we provide analogues for these three perspectives for relative monads. We show that while the first and third data still coincide, the second is no longer generally equivalent. It is weaker and not sufficient to provide semantics for RMM.

4.1 Connection with enriched relative monads

In higher-order languages, strong monads yield an operation $(A \rightarrow TB) \rightarrow (TA \rightarrow TB)$ providing an *internalisation* of the Kleisli extension into the language. Such internalisations are given semantics using *enriched category theory*. In this section, we connect our notion of strong relative monads (Definition 3.3) to enriched relative monads.

Definition 4.5 ([2]). Let $(\mathcal{V}, \otimes, I)$ be a symmetric monoidal category, $|\mathcal{A}|$ a collection of objects, $\mathcal{C}$ a $\mathcal{V}$ -enriched category, and $J : |\mathcal{A}| \rightarrow |\mathcal{C}|$ a mapping of objects. A $\mathcal{V}$ -enriched J -relative monad $(T, \eta, (-)^*)$, consists of:

- An object $TA \in |\mathcal{C}|$ for each $A \in |\mathcal{A}|$,
- A morphism $\eta_A : I \rightarrow \mathcal{C}[JA, TA]$ in $\mathcal{V}$ for each $A \in |\mathcal{A}|$,
- A family of morphisms $* : \mathcal{C}[JA, TB] \rightarrow \mathcal{C}[TA, TB]$ in $\mathcal{V}$

making the following diagrams commute.

$$\begin{array}{ccc}
 I & \xrightarrow{\eta_A} & C[JA, TA] \\
 \searrow \text{id}_{TA} & & \downarrow * \\
 & & C[TA, TA]
 \end{array}
 \qquad
 \begin{array}{ccc}
 C[JA, TB] \otimes I & \xrightarrow{* \otimes \eta_A} & C[TA, TB] \otimes C[JA, TA] \\
 \searrow \rho_{C[JA, TB]} & & \downarrow \bullet_{JA, TA, TB} \\
 & & C[JA, TB]
 \end{array}$$

$$\begin{array}{ccc}
 C[JB, TC] \otimes C[JA, TB] & \xrightarrow{* \otimes *} & C[TB, TC] \otimes C[TA, TB] \\
 \downarrow * \otimes C[JA, TB] & & \downarrow \bullet_{TA, TB, TC} \\
 C[TB, TC] \otimes C[JA, TB] & \xrightarrow{\bullet_{JA, TB, TC}} C[JA, TC] \xrightarrow{*} & C[TA, TC]
 \end{array}$$

A morphism of $\mathcal{V}$ -enriched J -relative monads $\gamma : (T, \eta, (-)^*) \rightarrow (S, \nu, (-)^\dagger)$ consists of, for each $A \in |\mathcal{A}|$, maps $\gamma_A : I \rightarrow C[TA, SA]$ satisfying compatibility conditions [2].

As in the unenriched setting, when $\mathcal{A}$ is an enriched category with objects $|\mathcal{A}|$ and $J : |\mathcal{A}| \rightarrow |\mathcal{C}|$ extends to an enriched functor $J : \mathcal{A} \rightarrow \mathcal{C}$, T extends to an enriched functor $T : \mathcal{A} \rightarrow \mathcal{C}$, where $T_{A,B} = * \circ \bullet_{JA, JB, TB} \circ (\eta_A \otimes C[JA, JB]) \circ \lambda_{C[JA, JB]}^{-1} \circ J_{A,B}$.

THEOREM 4.6. *Let $|\mathcal{A}|$ be a collection of objects, $\mathcal{C}$ a small symmetric monoidal category, and $J : |\mathcal{A}| \rightarrow |\mathcal{C}|$ a mapping on objects. Then there is an isomorphism of categories between:*

- (1) *The category of $(\widehat{C}, \widehat{\otimes}, \widehat{I})$ -enriched J -relative monads, via the γ -fully faithfully enrichment of $\mathcal{C}$.*
- (2) *The category of strong J -relative monads.*

PROOF. By the Yoneda lemma, there is an isomorphism $C[X, Y] \cong C(- \otimes X, Y)$. The bijection on the units of the monad is thus given by:

$$\widehat{C}(\widehat{I}, C[JA, TA]) \cong \widehat{C}(\gamma_{I_C}, C(- \otimes JA, TA)) \cong C(I_C \otimes JA, TA) \cong C(JA, TA)$$

The bijection on the Kleisli extensions is given by:

$$\widehat{C}(C(- \otimes JA, TB), C(- \otimes TA, TB)) \cong \widehat{C}(C[JA, TB], C[TA, TB])$$

The coherence conditions lift through these isomorphisms accordingly, and morphisms additionally correspond. $\square$

We recall that there is a forgetful functor $\mathcal{V}(I, -)_\# : \mathcal{V}\text{Cat} \rightarrow \text{Cat}$ defined by applying the functor $\mathcal{V}(I, -)$ to the $\mathcal{V}$ -objects of morphisms [33]. In the case of $\widehat{C}$ -enrichment, $\widehat{C}(\widehat{I}, -)$ is evaluation on I . The mapping from (1) to (2) in Theorem 4.6 can be understood to be applying this functor to the data of a $\widehat{C}$ -enriched relative monad, as it sends $(T, \eta, (-)^*) \mapsto (T, \eta_I(1_I), (-)_I^*)$. In particular, if $\mathcal{A}$ is an $\widehat{C}$ -enriched category with objects $|\mathcal{A}|$ and $\widehat{J}$ is an $\widehat{C}$ -enriched functor, this mapping sends an enriched $\widehat{J}$ -relative monad to a strong $\widehat{C}(\widehat{I}, -)_\# \widehat{J}$ -relative monad. Since J and $\mathcal{A}$ are not uniquely enriched, the correspondence in Theorem 4.6 does not extend to the data of the induced functor on T . However, if we fix a choice of enrichment, we recover the correspondence for the induced functors, partly extending Proposition 4.4.

COROLLARY 4.7. *Let $(C, \otimes, I)$ be a small symmetric monoidal category, $\mathcal{A}$ a $\widehat{C}$ -enriched category, and $\widehat{J} : \mathcal{A} \rightarrow \mathcal{C}$ a $\widehat{C}$ -enriched functor. Then, the category of $\widehat{C}$ -enriched $\widehat{J}$ -relative monads is isomorphic to the category of strong $\widehat{C}(\widehat{I}, -)_\# \widehat{J}$ -relative monads.*

Moreover, this isomorphism preserves the functor structure induced by $\widehat{J}$ and $\widehat{C}(\widehat{I}, -)_\# \widehat{J}$ for the enriched and strong relative monads, respectively.

Example 4.8. If J is a fully faithful functor, then the J -fully faithful enrichment of $\mathcal{A}$ and J gives a $\widehat{J}$ satisfying $J = \widehat{C}(\widehat{I}, -)_{\#} \widehat{J}$ and so Corollary 4.7 applies.

There is a left adjoint $F_{\widehat{C}} \dashv \widehat{C}(\widehat{I}, -)_{\#}$, given on objects by sending a category $\mathcal{A}$ to a $\widehat{C}$ -enriched category with the same objects and with $(F_{\widehat{C}}\mathcal{A})[A, B] = \mathcal{A}(A, B) \times C(-, I)$. Hence, functors $J : \mathcal{A} \rightarrow C$ correspond to $\widehat{C}$ -enriched functors $\widehat{J} : F_{\widehat{C}}\mathcal{A} \rightarrow C$, and so we can obtain a similar result in the other direction.

COROLLARY 4.9. *Let C be a small symmetric monoidal category, and let $J : \mathcal{A} \rightarrow C$ be any functor and $\widehat{J} : F_{\widehat{C}}\mathcal{A} \rightarrow C$ be the transpose under the adjunction $F_{\widehat{C}} \dashv \widehat{C}(\widehat{I}, -)_{\#}$. Then, the category of $\widehat{C}$ -enriched $\widehat{J}$ -relative monads is isomorphic to the category of strong J -relative monads.*

Moreover, when the relative monads are given the functor structure induced by $\widehat{J}$ and J , respectively, this isomorphism coincides with the transpose under the adjunction $F_{\widehat{C}} \dashv \widehat{C}(\widehat{I}, -)_{\#}$.

Example 4.10. Since $J = \widehat{C}(\widehat{I}, -)_{\#} \widehat{J} \circ \iota_{\mathcal{A}}$, where ι is the unit of the adjunction $F_{\widehat{C}} \dashv \widehat{C}(\widehat{I}, -)_{\#}$, we cannot directly obtain the functor structure of any strong J -relative monad by applying the forgetful functor $\widehat{C}(\widehat{I}, -)_{\#}$. However, if I is initial or terminal, or more generally $C(I, I) = 1$, the unit of the adjunction $F_{\widehat{C}} \dashv \widehat{C}(\widehat{I}, -)_{\#}$ is an isomorphism. Then, the correspondence in Theorem 4.6 extends to functors via Corollary 4.9.

When a category has enough structure, rather than internalising the Kleisli extension in its category of presheaves, we can internalise the structure directly.

THEOREM 4.11. *Let C be a symmetric monoidal closed category. Then C is self-enriched, and there is an isomorphism of categories between the category of C -enriched J -relative monads and the category of strong J -relative monads.*

PROOF. We can choose a suitable universe of sets so that C is small. The Yoneda embedding $y : C \rightarrow \widehat{C}$ is a fully faithful, strong closed symmetric monoidal functor. In particular, $yX \multimap yY \cong y(X \multimap Y)$, and the data of a $\widehat{C}$ -enriched J -relative monad unravels via the Yoneda lemma to the data of a C -enriched J -relative monad. The result then follows from Theorem 4.6. $\square$

4.2 Refined notions of strengths for relative monads

The notion of strength introduced in Definition 3.3 validates the following rule of RMM.

$$\frac{\Gamma \vdash u : TA \quad \Gamma, x : JA \vdash t : TB}{\Gamma \vdash \text{do } x \leftarrow u \text{ in } t : TB} \text{(T-bind)}$$

This allows us to sequence computations in arbitrary contexts Γ . However, for some languages using the relative monadic metalanguage framework, it is sufficient to validate **T-bind** for restricted contexts. For example, the nominal language for local state discussed in Section 3.2 only requires **T-bind** for contexts of shape $\Gamma = x_1 : JA_1, \dots, x_n : JA_n$. Furthermore, in Section 6, we show that the arrow calculus [43] is also a relative monadic language that restricts the shape of the context for sequencing computations. This motivates the following definition, which validates **T-bind** for contexts of shape $\Gamma = x_1 : WM_1, \dots, x_n : WM_n$.

Definition 4.12. Let $|\mathcal{A}|$ be a collection of objects, C and $\mathcal{M}$ be symmetric monoidal categories, and $J : |\mathcal{A}| \rightarrow |C|$ be a mapping of objects. If $(W, \kappa, \iota) : \mathcal{M} \rightarrow C$ is a strong symmetric monoidal functor [44], a W -strong J -relative monad consists of:

- An object $TA \in |C|$ for each $A \in |\mathcal{A}|$
- A morphism $\eta_A : JA \rightarrow TA$ for each $A \in |\mathcal{A}|$

- A collection of maps $(-)^* : C(W\Gamma \otimes JA, TB) \rightarrow C(W\Gamma \otimes TA, TB)$ natural in $\Gamma \in |\mathcal{M}|$

all such that for $f : W\Gamma \otimes JA \rightarrow TB$ and $g : W\Delta \otimes JB \rightarrow TC$, the following equations hold:

$$(\eta_A \circ \lambda_{JA} \circ (\iota^{-1} \otimes JA))^* = \lambda_{TA} \circ (\iota^{-1} \otimes TA) \quad f^* \circ (W\Gamma \otimes \eta_A) = f$$

$$g^* \circ (W\Delta \otimes f^*) \circ \alpha_{W\Delta, W\Gamma, TA} \circ (\kappa_{\Delta, \Gamma}^{-1} \otimes TA) = (g^* \circ (W\Delta \otimes f) \circ \alpha_{W\Delta, W\Gamma, JA} \circ (\kappa_{\Delta, \Gamma}^{-1} \otimes JA))^*$$

A *morphism of W -strong J -relative monads* $\gamma : (T, \eta, (-)^*) \rightarrow (S, \nu, (-)^\dagger)$ consists of, for each $A \in |\mathcal{A}|$, maps $\gamma_A : TA \rightarrow SA$ satisfying, for any $f : W\Gamma \otimes JA \rightarrow TB$:

$$\nu_A = \gamma_A \circ \eta_A \quad \gamma_B \circ f^* = (\gamma_B \circ f)^\dagger \circ (W\Gamma \otimes \gamma_A)$$

We write $\mathbf{WSMon}(J)$ for the category of W -strong J -relative monads.

Example 4.13. When $\mathcal{M} = C$ and $W = \text{id}_C$, we recover the definition of strong relative monad in Definition 3.3.

As with Definition 3.3, a W -strong J -relative monad has an underlying J -relative monad $(T, \eta, (-)^*)$, given by $f^{*U} = (f \circ \lambda_{JA} \circ (\iota^{-1} \otimes JA))^* \circ (\iota \otimes TA) \circ \lambda_{TA}^{-1}$ for $f \in C(JA, TB)$, which defines the object mapping for a forgetful functor $\mathbf{WSMon}(J) \rightarrow \mathbf{Mon}(J)$. Furthermore, there is a forgetful functor $U_W : \mathbf{SMon}(J) \rightarrow \mathbf{WSMon}(J)$ that provides a factorisation for the forgetful functor $\mathbf{SMon}(J) \rightarrow \mathbf{Mon}(J)$. Under additional conditions, U_W is an isomorphism and so W -strong J -relative monads coincide with strong J -relative monads.

PROPOSITION 4.14. *Assume that C is a symmetric monoidal category such that $X \otimes -$ preserves colimits for all $X \in |C|$. If $W : \mathcal{M} \rightarrow C$ is a dense strong symmetric monoidal functor, then $U_W : \mathbf{SMon}(J) \rightarrow \mathbf{WSMon}(J)$ is an isomorphism of categories.*

PROOF. Given a W -strong J -relative monad $(T, \eta, (-)^*)$, we define a strong J -relative monad $(T, \eta, (-)^\dagger)$ by

$$(-)^\dagger : C(X \otimes JA, TB) \cong \lim_{M \in (W \downarrow X)} C(WM \otimes JA, TB) \rightarrow \lim_{M \in (W \downarrow X)} C(WM \otimes TA, TB) \cong C(X \otimes TA, TB),$$

where $(W \downarrow X)$ is a comma category [44]. One can check that this provides an inverse to the restriction map defined by U_W , and that the strong relative monad laws follow. $\square$

Remark. As noted in Section 2, relative monads are often studied for *well-behaved* roots J . In this case, for any J -relative monad T , the left Kan extension $\text{Lan}_J T$ [44] exists and receives a monad structure which makes it initial in the comma category $(T \downarrow \text{res}_J)$ [1]. Under the additional conditions of Proposition 4.14, with $W = J$, this result extends to *strong* relative monads. In particular, if T is a J -strong J -relative monad, then $\text{Lan}_J T$ has a strong monad structure which makes it initial in the comma category $(T \downarrow \text{pres}_J)$.

If $\mathcal{A}$ is additionally a symmetric monoidal category and J is a strong symmetric monoidal functor, we can recover the full correspondence of Proposition 4.4 for J -strong J -relative monads. In this case, we can equivalently express the strength via maps $\theta_{A,B} : JA \otimes TB \rightarrow T(A \otimes B)$. This coincides with previous definitions of strong relative monad [65], and is also closer to the historical presentation [36].

Definition 4.15. Let $\mathcal{A}$ and C be symmetric monoidal categories and $(J, \kappa, \iota) : \mathcal{A} \rightarrow C$ a strong symmetric monoidal functor. A *strength map* for a J -relative monad $(T, \eta, (-)^*)$ consists of morphisms

$$\theta_{A,B} : JA \otimes TB \rightarrow T(A \otimes B),$$

natural in $A, B \in |\mathcal{A}|$, that make the following diagrams commute [65].

$$\begin{array}{ccc}
 I \otimes TB & \xrightarrow{\lambda_{TB}} & TB \\
 \downarrow \iota \otimes TB & & \uparrow T\lambda_B \\
 JI \otimes TA & \xrightarrow[\theta_{I,B}]{} & T(I \otimes B)
 \end{array}
 \quad
 \begin{array}{ccc}
 (JA \otimes JB) \otimes TC & \xrightarrow{\kappa_{A,B} \otimes TC} & J(A \otimes B) \otimes TC \xrightarrow{\theta_{A \otimes B, C}} T((A \otimes B) \otimes C) \\
 \downarrow \alpha_{JA, JB, TC} & & \downarrow T\alpha_{A, B, C} \\
 JA \otimes (JB \otimes TC) & \xrightarrow[\theta_{B, C}]{} & JA \otimes T(B \otimes C) \xrightarrow[\theta_{A, B \otimes C}]{} T(A \otimes (B \otimes C))
 \end{array}$$

$$\begin{array}{ccc}
 JA \otimes JB & \xrightarrow{\kappa_{A,B}} & J(A \otimes B) \\
 \downarrow JA \otimes \eta_B & & \downarrow \eta_{A \otimes B} \\
 JA \otimes Tb & \xrightarrow[\theta_{A,B}]{} & T(a \otimes B)
 \end{array}
 \quad
 \begin{array}{ccc}
 JA \otimes TB & \xrightarrow{\theta_{A,B}} & T(A \otimes B) \\
 \downarrow JA \otimes f^* & & \downarrow (\theta_{A,C} \circ (JA \otimes f) \circ \kappa_{A,B}^{-1})^* \\
 JA \otimes TC & \xrightarrow[\theta_{A,C}]{} & T(A \otimes C)
 \end{array}$$

A morphism $\gamma : (T, \eta, (-)^*) \rightarrow (S, \nu, (-)^\dagger)$ of J -relative monads is said to *preserve strength* if

$$\gamma_{A \otimes B} \circ \theta_{A,B} = \theta'_{A,B} \circ (JA \otimes \gamma_B)$$

PROPOSITION 4.16. *Let $\mathcal{A}$ and $\mathcal{C}$ be symmetric monoidal categories and $(J, \kappa, \iota) : \mathcal{A} \rightarrow \mathcal{C}$ a strong symmetric monoidal functor. The following categories are isomorphic:*

- (1) *The category of J -strong J -relative monads*
- (2) *The category of J -relative monads with strength maps and morphisms preserving strength*
- (3) *The category of $(\widehat{\mathcal{A}}, \widehat{\otimes}, \widehat{I})$ -enriched $\widehat{J}$ -relative monads, where*
 - $\mathcal{A}$ *is given the γ -fully faithful $\widehat{\mathcal{A}}$ -enrichment*
 - $\mathcal{C}$ *is enriched in $\widehat{\mathcal{A}}$ by $C[X, Y] = C(J- \otimes X, Y)$*
 - $\widehat{J}$ *is the enriched functor with components*

$$\mathcal{A}[A, B] \cong \mathcal{A}(- \otimes A, B) \rightarrow \mathcal{C}(J(- \otimes A), JB) \cong \mathcal{C}(J- \otimes JA, JB) \cong \mathcal{C}[JA, JB].$$

PROOF. For the equivalence of (1) and (2), if $(T, \eta, (-)^*)$ is a J -strong J -relative monad, then

$$\theta_{A,B} = (\eta_{A \otimes B} \circ \kappa_{A,B})^*$$

define strength maps for the underlying relative monad of $(T, \eta, (-)^*)$. (See also the similar result in [59].) Conversely, given strength maps, we define

$$f^\dagger = (f \circ \kappa_{A,B}^{-1})^* \circ \theta_{A,B}$$

for any $f : JA \otimes JB \rightarrow TC$. One can check that this defines a J -strong J -relative monad, and that these mappings are mutual inverses. For the equivalence of (1) and (3), the data provides an enrichment of $\mathcal{C}$ and J , using the fact that J is strong symmetric monoidal. The correspondence on units is given by

$$\widehat{\mathcal{A}}(\widehat{I}, C[JA, TA]) \cong \widehat{\mathcal{A}}(\widehat{I}, C(J- \otimes JA, TA)) \cong C(JI \otimes JA, TA) \cong C(JA, TA),$$

and on the extensions by

$$\widehat{\mathcal{A}}(C[JA, TB], C[TA, TA]) = \widehat{\mathcal{A}}(C(J- \otimes JA, TB), C(J- \otimes TA, TB)).$$

The coherence conditions lift through these isomorphisms. $\square$

One motivating example for J -strengths is when J is the Yoneda embedding $\gamma : \mathcal{C} \rightarrow \widehat{\mathcal{C}}$ of a small category $\mathcal{C}$, as in Example 2.6. Then a J -strong J -relative monad is a *strong promonad* [65] or a notion of *arrow* [25]. In this case, J -strong J -relative monads, in fact, coincide with the notion of strong J -relative monad in Definition 3.3.

COROLLARY 4.17. *The Yoneda embedding satisfies the conditions of Proposition 4.14, and so $U_Y : \mathbf{SMon}(Y) \rightarrow Y\mathbf{SMon}(Y)$ is an isomorphism of categories.*

5 Encoding graded monads in the linear-non-linear relative monadic metalanguage

Graded monads [30] are a semantic tool for analysing the usage of effects. They stratify computation via annotation with grades, permitting a more fine-grained analysis of programs. Graded monads have been used for memory and output analysis for the state and writer monad, non-determinism with cuts [30, 31], and to encode epistemic uncertainty and programmable inference in probabilistic programs [41, 42]. Graded monads are also understood to be instances of enriched relative monads [48], and in Theorem 5.5, we show that strong graded monads can be understood as strong relative monads. We exploit this semantic correspondence to refine RMM to a term calculus for graded monads, based on the linear-non-linear calculus [7].

Definition 5.1 ([30]). Let $(\mathcal{M}, \oplus, e)$ be a symmetric monoidal category. A *strong $\mathcal{M}$ -graded monad* on a Cartesian category $(\mathcal{A}, \times, 1)$ consists of

- Functors $T_{(-)}A : \mathcal{M}^{\text{op}} \rightarrow \mathcal{A}$ for each $A \in |\mathcal{A}|$
- A unit map $\eta_A : A \rightarrow T_e A$ for each $A \in |\mathcal{A}|$
- A collection of maps $(-)^*_{m,n} : \mathcal{A}(\Gamma \times A, T_n B) \rightarrow \mathcal{A}(\Gamma \times T_m A, T_{m \oplus n} B)$ natural in $\Gamma \in |\mathcal{A}|$ and $n \in |\mathcal{M}|$ and extranatural in $m \in |\mathcal{M}|$

such that for any $f : \Gamma \times A \rightarrow T_m B$ and $g : \Gamma \times B \rightarrow T_n C$ the following equations hold,

$$\begin{aligned} (\eta_A \circ \pi_A)^*_{m,e} &= T_{\rho_m} A \circ \pi_{T_m A}, \\ (f)^*_{e,m} \circ (\Gamma \times \eta_A) &= T_{\lambda_m} B \circ f, \\ (g)^*_{l \oplus m, n} \circ (\pi_\Gamma, (f)^*_{l,m}) &= T_{\alpha_{l,m,n}} C \circ ((g)^*_{m,n} \circ (\pi_\Gamma, f))^*_{l, m \oplus n}, \end{aligned}$$

where $\pi_\Gamma : \Gamma \times A \rightarrow \Gamma$ and $\pi_A : \Gamma \times A \rightarrow A$ are the projections.

Example 5.2. The list monad on **Set** is a strong $(\mathbb{N}, \geq, \cdot, 1)$ graded monad, where $T_n X = \{\vec{x} \in X^* \mid \text{length}(\vec{x}) \leq n\}$.

Monadic style programming can be refined into a graded monadic metalanguage (GMM), by grading the computation types by a symmetric monoidal category $\mathcal{M}$ [30]. The language $\text{GMM}(\mathcal{M})$ is defined as follows.

$$\begin{array}{ll} \text{Types:} & A, B ::= 1 \mid A \times B \mid T_m A \quad (m \in |\mathcal{M}|) \\ \text{Contexts:} & \Gamma ::= \emptyset \mid \Gamma, x : X \end{array}$$

$\text{GMM}(\mathcal{M})$ has standard Cartesian term constructors along with the following terms for graded monadic sequencing:

$$\frac{\Gamma \vdash u : A}{\Gamma \vdash \text{return } u : T_e A} \quad \frac{\Gamma \vdash u : T_m A \quad \Gamma, x : A \vdash t : T_n B}{\Gamma \vdash \text{do } x \leftarrow u \text{ in } t : T_{m \oplus n} B} \quad \frac{\Gamma \vdash u : T_n A}{\Gamma \vdash T_\xi A u : T_m A} (\xi \in \mathcal{M}(m, n))$$

In particular, `return` assigns a value at the unit grading, while sequencing computations tensors the grades of the individual computations. These terms satisfy associativity and unitality equations (see Appendix B.2 for the full equational theory and term calculus).

Definition 5.3. A model of $\text{GMM}(\mathcal{M})$ consists of

- A Cartesian category $\mathcal{C}$
- A strong $\mathcal{M}$ -graded monad

The graded monadic metalanguage was introduced in a richer setting with sum types [30], and the models of that type theory have additional structure. However, the same soundness results apply in our restricted setting, and we obtain completeness.

THEOREM 5.4 ([30]). *The models of $GMM(\mathcal{M})$ provide a sound and complete semantics.*

5.1 Strong graded monads as strong relative monads

The data of an $\mathcal{M}$ -graded monad is in bijective correspondence with the data for an $\widehat{\mathcal{M}}$ -enriched relative monad for a suitably chosen root [48, Thm. 1]. In Theorem 5.5, we extend this to strong graded monads. We begin by explicitly describing this connection for $\mathcal{M}$ -graded monads on **Set**, which are always strong. In this case, the enrichment of [48, Thm. 1] is equivalent to strength of the relative monad by Theorem 4.11. Hence, we can understand $\mathcal{M}$ -graded monads on **Set** as strong J -relative monads, where $J : \mathbf{Set} \rightarrow \widehat{\mathcal{M}}$ is defined by

$$JX = \mathcal{M}(-, e) \cdot X,$$

with $Y \cdot X$ denoting the copower of X by Y and $\widehat{\mathcal{M}}$ is considered with its Day monoidal structure. We will now explicitly describe how this corresponds to a strong graded monad. Note that J has a right adjoint, $J \dashv R$, where R is given by evaluation on e , since

$$\widehat{\mathcal{M}}(JX, F) \cong \mathbf{Set}(X, \widehat{\mathcal{M}}(\mathcal{M}(-, e), F(-))) \cong \mathbf{Set}(X, Fe).$$

Hence, the correspondence on the data of the units is via the adjunction:

$$\widehat{\mathcal{M}}(JX, TY) \cong \mathbf{Set}(X, T_e Y)$$

Let $W : \mathcal{M} \times \mathbf{Set} \rightarrow \widehat{\mathcal{M}}$ be defined by $W(m, \Gamma) = J\Gamma \otimes \mathcal{M}(-, m)$. Then, the following diagram gives a correspondence between extension operators for a strong $\mathcal{M}$ -graded monad on **Set** and extension operators for a W -strong J -relative monad.

$$\begin{array}{ccccc} \mathbf{Set}(\Gamma \times X, T_n Y) & \cong & \widehat{\mathcal{M}}(J(\Gamma \times X), \mathcal{M}(-, n) \multimap TY) & \cong & \widehat{\mathcal{M}}((J\Gamma \otimes \mathcal{M}(-, n)) \otimes JX, TY) \\ \downarrow (=)^*_{-,n} & & & & \downarrow (=)^* \\ \widehat{\mathcal{M}}(\Gamma \times T_- X, T_{-\oplus n} Y) & \cong & \widehat{\mathcal{M}}(J\Gamma \otimes TX, \mathcal{M}(-, n) \multimap TY) & \cong & \widehat{\mathcal{M}}((J\Gamma \otimes \mathcal{M}(-, n)) \otimes TX, TY) \end{array}$$

Note that we have used the natural isomorphisms $Fn \cong R(\mathcal{M}(-, n) \multimap F)$ and $(J\Gamma \otimes F)(m) \cong X \times Fm$, which hold for any presheaf $F : \mathcal{M}^{\text{op}} \rightarrow \mathbf{Set}$, and that J is strong monoidal. By Proposition 4.14, this also corresponds to the data for a strong J -relative monad. We now generalise this example to any Cartesian closed category with small limits and colimits.

THEOREM 5.5. *Let C be a Cartesian closed category with all small limits and colimits, and $\mathcal{M}$ a small symmetric monoidal category. Let $\overline{\mathcal{M}} = [\mathcal{M}^{\text{op}}, C]$ and define the functor $J : C \rightarrow \overline{\mathcal{M}}$ by*

$$JX = \mathcal{M}(-, e) \cdot X$$

Then there is a bijective correspondence between strong J -relative monads on $(\overline{\mathcal{M}}, \overline{\otimes}, \mathcal{M}(-, e) \cdot 1)$ and strong $\mathcal{M}$ -graded monads on C , where $\overline{\otimes}$ is the C -enriched Day convolution [14]:

$$(F \overline{\otimes} G)(m) = \int^{n, p \in \mathcal{M}} \mathcal{M}(m, n \oplus p) \cdot (Fn \times Gp)$$

PROOF. The C -enriched Day convolution for the free C -enrichment of $\mathcal{M}$ defines a symmetric monoidal closed structure on $\overline{\mathcal{M}}$. J has a right adjoint R given by evaluation on e and is strong monoidal. The isomorphisms described above for **Set** apply in the general case, giving the correspondence between units and the extension operators of a strong graded monad and a W -strong

J -relative monad, where $W : \mathcal{M} \times C \rightarrow \overline{\mathcal{M}}$ is given by $W(m, X) = \mathcal{M}(-, m) \cdot X$. Proposition 4.14 does not directly apply, but its argument can be generalised to give extension operators

$$\overline{\mathcal{M}}(\Gamma \otimes JX, TY) \cong \int_n \overline{\mathcal{M}}(W(n, \Gamma n) \otimes JX, TY) \rightarrow \int_n \overline{\mathcal{M}}(W(n, \Gamma n) \otimes TX, TY) \cong \overline{\mathcal{M}}(\Gamma \otimes TX, TY),$$

since W is enriched dense. $\square$

5.2 The linear-non-linear RMM

The linear-non-linear calculus (LNL) [7] is a language that accommodates both linear and non-linear programs. It provides a type system that incorporates *resource sensitivity* via its linear part, with the expressivity of the simply typed lambda calculus, which allows functions such as $\lambda(x : \text{int}).x + x$. *Grades* can be interpreted as cost-sensitive computations, so by this analogy and Theorem 5.5, we integrate LNL and RMM to produce a graded monadic metalanguage that is a conservative extension of GMM. The LNL calculus consists of the following types:

$$\begin{aligned} \mathcal{A} \text{ types:} \quad & A, B ::= 1 \mid A \times B \mid A \rightarrow B \mid RX \\ \mathcal{C} \text{ types:} \quad & X, Y ::= I \mid X \otimes Y \mid X \multimap Y \mid JA \\ \mathcal{A} \text{ contexts:} \quad & \Gamma, ::= \emptyset \mid \Gamma, a : A \\ \mathcal{C} \text{ contexts:} \quad & \Delta ::= \emptyset \mid \Delta, x : X \end{aligned}$$

There is a *non-linear $\mathcal{A}$ -judgement* $\Gamma \vdash_{\mathcal{A}} u : A$ and a *mixed linear-non-linear $\mathcal{C}$ -judgement* $\Gamma; \Delta \vdash_{\mathcal{C}} t : X$. Beyond the terms for the simply typed lambda calculus for $\mathcal{A}$ types and terms for the linear lambda calculus for $\mathcal{C}$ types (Appendix B.3), there are linear-non-linear interaction terms.

$$\begin{array}{c} \frac{\Gamma \vdash_{\mathcal{A}} u : A}{\Gamma; \emptyset \vdash_{\mathcal{C}} J(u) : JA} \qquad \frac{\Gamma; \emptyset \vdash_{\mathcal{C}} t : X}{\Gamma \vdash_{\mathcal{A}} R(t) : RX} \\[10pt] \frac{\Gamma; \Delta_1 \vdash_{\mathcal{C}} t : JA \quad \Gamma, a : A; \Delta_2 \vdash_{\mathcal{C}} s : X}{\Gamma; \Delta_1, \Delta_2 \vdash_{\mathcal{C}} \text{let } J(a) \text{ be } t \text{ in } s : X} \qquad \frac{\Gamma \vdash_{\mathcal{A}} u : RX}{\Gamma; \emptyset \vdash_{\mathcal{C}} \text{derelict } u : X} \end{array}$$

The following models provide a sound semantics for LNL [7] with the equations in Appendix B.3.

Definition 5.6. An LNL model consists of

- A Cartesian closed category $\mathcal{A}$
- A symmetric monoidal closed category $\mathcal{C}$
- A strong symmetric monoidal functor $J : \mathcal{A} \rightarrow \mathcal{C}$ with a right adjoint $J \dashv R$

Example 5.7. If $\mathcal{C}$ is a Cartesian closed category with all small limits and colimits, and $\mathcal{M}$ is a small symmetric monoidal category, then the triple $(\mathcal{C}, \overline{\mathcal{M}}, J)$ given in Theorem 5.5 is an LNL model. The right adjoint R is given by evaluation on e .

In view of this example, Theorem 5.5 shows that we can obtain a graded monad on this LNL model from any strong J -relative monad. In fact, this construction applies to any LNL model. We extend LNL accordingly, with a strong relative monad and additional *grade types* $\underline{m}$ which allow the interpretation of the objects of $\mathcal{M}$ inside $\mathcal{C}$. This defines a new language LNL-RMM($\mathcal{M}$) for $\mathcal{M}$ -graded monads, defined as follows.

$$\begin{aligned} \mathcal{A} \text{ types:} \quad & A, B ::= 1 \mid A \times B \mid A \rightarrow B \mid RX \\ \mathcal{C} \text{ types:} \quad & X, Y ::= I \mid \underline{m} \mid X \otimes Y \mid X \multimap Y \mid JA \mid TA \qquad (m \in |\mathcal{M}|) \end{aligned}$$

The linear RMM terms are given by:

$$\frac{\Gamma; \Delta \vdash_C t : JA}{\Gamma; \Delta \vdash_C \text{return } t : TA} \quad \frac{\Gamma; \Delta_1 \vdash_C t : TA \quad \Gamma; \Delta_2, x : JA \vdash_C s : TB}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{do } x \leftarrow t \text{ in } s : TB}$$

The embedding of the grade types is given by:

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m}}{\Gamma; \Delta \vdash_C \underline{\xi} t : \underline{n}} (\xi \in \mathcal{M}(m, n)) \quad \frac{\Gamma; \Delta \vdash_C t : \underline{e}}{\Gamma; \Delta \vdash_C \text{unmerge } t : I} \quad \frac{\Gamma; \Delta \vdash_C t : I}{\Gamma; \Delta \vdash_C \text{merge } t : \underline{e}}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m} \oplus \underline{n}}{\Gamma; \Delta \vdash_C \text{unmerge } t : \underline{m} \otimes \underline{n}} \quad \frac{\Gamma; \Delta \vdash_C t : \underline{m} \otimes \underline{n}}{\Gamma; \Delta \vdash_C \text{merge } t : \underline{m} \oplus \underline{n}}$$

The full term calculus and its equations are given in Appendix B.3. We can now define a strong $\mathcal{M}$ -graded monad on the $\mathcal{A}$ -types in the term calculus of LNL-RMM($\mathcal{M}$). The graded monad type constructor $T_m A$ on $\mathcal{A}$ is given by:

$$T_m A = R(\underline{m} \multimap TA) \quad (5)$$

The unit of the graded monad is then:

$$\lambda(a : A). R(\lambda(x : \underline{e}). \text{let } () \text{ be unmerge } x \text{ in return } J(a)) : A \rightarrow T_e A,$$

The bind $\gg= : T_m A \rightarrow (A \rightarrow T_n B) \rightarrow T_{m \oplus n} B$ is:

$$\lambda(f : T_m A). \lambda(g : A \rightarrow T_n B). R(\lambda(s : \underline{m} \oplus \underline{n}). \text{let } (t, r) \text{ be unmerge } s \text{ in} \\ (\text{do } x \leftarrow (\text{derelict } f) t \text{ in } (\text{let } J(a) \text{ be } x \text{ in } (\text{derelict } g a) r)))$$

For $\xi : m \rightarrow n$, the regrading morphism $T_\xi X : T_n X \rightarrow T_m X$ is given by:

$$\lambda(f : T_n X). R(\lambda(s : \underline{m}). (\text{derelict } f) (\underline{\xi} s))$$

Example 5.8. If a symmetric monoidal category $\mathcal{M}$ is *finitely generated*, then we can simplify the signature of LNL-RMM($\mathcal{M}$) by only adding generator grade types, and encoding the relations in the equational theory. This makes LNL-RMM($\mathcal{M}$) suitable for implementations of a graded monad. For example, an $(\mathbb{N}, \geq, +, 0)$ -graded monad can be specified by the following type signature.

$$\begin{aligned} \mathcal{A} \text{ types: } & A, B ::= 1 \mid A \times B \mid A \rightarrow B \mid RX \\ \mathcal{C} \text{ types: } & X, Y ::= I \mid \underline{1} \mid X \otimes Y \mid X \multimap Y \mid JA \mid TA \end{aligned}$$

The additional terms and equations required are:

$$\frac{}{\Gamma; \emptyset \vdash_C * : \underline{1}} \quad \frac{\Gamma; \emptyset \vdash_C s : \underline{m} \multimap \underline{n} \quad \Gamma; \emptyset \vdash_C t : \underline{m} \multimap \underline{n}}{\Gamma; \emptyset \vdash_C s = t : \underline{m} \multimap \underline{n}}$$

where $\underline{m}$ is syntactic sugar for the m -fold product $\underline{1} \otimes \cdots \otimes \underline{1}$.

We can illustrate the utility of the linear and non-linear aspects of the language for graded monads via two example programs.

Example 5.9. The non-linearity of the term calculus for $\mathcal{A}$ types allows us to define a term that self-sequences an effectful function, which would not be possible in a purely linear language.

$$\lambda(a : A). \lambda(f : A \rightarrow T_m A). (f a \gg= f) : A \rightarrow T_{m \oplus m} A$$

Example 5.10. The linearity of the term calculus for C -types provides a suitable constraint for reasoning about grade types. Linear type theories restrict which terms are derivable by preventing the discarding and duplication of variables. Consider a program in LNL-RMM,

$$\text{runBoth} : T_m A \times T_m B \rightarrow T_{m \oplus m} (A \times B),$$

which runs each computation of cost m , returning the paired result necessarily of cost $m \oplus m$. If we extended the C term derivations to a non-linear type theory, allowing the discarding and duplication of variables, then we could duplicate $\underline{m}$, now permitting the following program:

$$\text{duplicateAndRunBoth} : T_m A \times T_m B \rightarrow T_m (A \times B)$$

Hence, the linearity restriction on derivable terms for C -types facilitates effect-usage-sensitive computations.

The strong graded monad on $\mathcal{A}$ -types defines an embedding of $\text{GMM}(\mathcal{M})$ into the term calculus of $\text{LNL-RMM}(\mathcal{M})$. This means that there is a map on $\text{GMM}(\mathcal{M})$ types and terms (denoted $[-]$) into the $\mathcal{A}$ -types and derivable terms of $\text{LNL-RMM}(\mathcal{M})$ that preserves the equational theory of $\text{GMM}(\mathcal{M})$. We will show that this embedding is *conservative*, so that it additionally *reflects* the equational theory of $\text{GMM}(\mathcal{M})$:

$$\Gamma \vdash u = v : A \iff [\Gamma] \vdash_{\mathcal{A}} [u] = [v] : [A]$$

We begin by identifying the models of $\text{LNL-RMM}(\mathcal{M})$.

Definition 5.11. A model of $\text{LNL-RMM}(\mathcal{M})$ is an LNL model $(\mathcal{A}, C, J \dashv R)$ along with:

- A strong symmetric monoidal functor $\mathcal{M} \rightarrow C$ which we denote by $m \mapsto \underline{m}$
- A strong J -relative monad $(T, \eta, (-)^*)$

THEOREM 5.12. *The models of $\text{LNL-RMM}(\mathcal{M})$ are a sound semantics.*

PROOF. The construction of the type theory is modular; i.e. we add terms from linear RMM with no additional equations with the terms of LNL. A model of $\text{LNL-RMM}(\mathcal{M})$ is therefore a model of LNL and (linear) RMM. One can also directly verify that the additional terms and equations for the grade types $\mathcal{M}$ are validated by a strong symmetric monoidal functor. $\square$

In categorical semantics, it is common to show completeness of a class of models by constructing a term model from the types, terms and equations. We are not aware of a proof of completeness of LNL in this fashion for the models in Definition 5.6, and that result is not within the scope of this paper. It is worth noting that due to the modularity of our type theory, a completeness result for $\text{LNL-RMM}(\mathcal{M})$ would follow from the completeness of LNL. However, the soundness of $\text{LNL-RMM}(\mathcal{M})$ (Theorem 5.12) combined with the completeness of $\text{GMM}(\mathcal{M})$ (Theorem 5.4), is sufficient for us to show the conservativity of the interpretation of graded monads in $\text{LNL-RMM}(\mathcal{M})$.

THEOREM 5.13. *The interpretation of $\text{GMM}(\mathcal{M})$ types and terms in the term calculus of $\text{LNL-RMM}(\mathcal{M})$ is a conservative extension.*

PROOF. Suppose $(\mathcal{A}, T)$ is a small model of $\text{GMM}(\mathcal{M})$ (where T denotes a strong $\mathcal{M}$ -graded monad). A strong $\mathcal{M}$ -graded monad on a small Cartesian category $\mathcal{A}$ can equivalently be given by

- Endofunctors $T_m : C \rightarrow \mathcal{A}$
- Maps $(\theta_m)_{A,B} : A \otimes T_m B \rightarrow T_m (A \otimes B)$ natural in $A, B \in |\mathcal{A}|$
- Natural transformations $T_\xi : T_n \rightarrow T_m$ for each $\xi : m \rightarrow n$
- A natural transformation $\eta : \text{id}_{\mathcal{A}} \rightarrow T_e$
- Natural transformations $\mu_{m,n} : T_m T_n \rightarrow T_{m \oplus n}$

satisfying equations [30]. By applying the 2-functor $\mathcal{A} \mapsto \widehat{\mathcal{A}}$ to this data, we obtain a strong $\mathcal{M}$ -graded monad on $C = \widehat{\mathcal{A}}$. We thus have a model $(C, \widehat{T})$ of $\text{GMM}(\mathcal{M})$ and a full and faithful functor $\gamma : \mathcal{A} \rightarrow C$ which strictly preserves the interpretation of $\text{GMM}(\mathcal{M})$ types and terms. Now, since C is a Cartesian closed category with all small limits and colimits, by Example 5.7, the adjunction described in Theorem 5.5 provides an LNL model. Furthermore, since $\widehat{T}$ is a strong graded monad on C , by the correspondence in Theorem 5.5, we obtain a model of $\text{LNL-RMM}(\mathcal{M})$, where the strong symmetric monoidal functor $\mathcal{M} \rightarrow \overline{\mathcal{M}}$ is given by the enriched Yoneda embedding $\underline{m} = \mathcal{M}(m, -) \cdot 1$. One can also verify that the correspondence in Theorem 5.5 yields (up to isomorphism) the strong graded monad on C , defined by $\llbracket R(m \multimap TX) \rrbracket$. Hence, any small model of $\text{GMM}(\mathcal{M})$ embeds fully faithfully into a model of $\text{LNL-RMM}(\mathcal{M})$, and so by letting $\mathcal{A}$ be the term model of $\text{GMM}(\mathcal{M})$, we see that the embedding is a conservative extension. $\square$

6 The arrow calculus in the relative monadic metalanguage

Arrows are programming constructs that generalise monads to computations that are not strictly sequential [25]. A calculus for arrows and corresponding semantics has been developed over the years [5, 25, 52, 58], culminating in *the arrow calculus* [43], which has a strong resemblance to the RMM framework introduced in this paper, with analogous unit and bind terms. On the semantic side, this is not surprising since arrows have been described as strong monads relative to the Yoneda embedding (see [65] and Section 4.2). The arrow calculus extends the simply typed lambda calculus with a binary type constructor $\multimap$.

Types: $A, B ::= 1 \mid A \times B \mid A \rightarrow B \mid A \multimap B$

Contexts: $\Gamma, \Delta ::= \emptyset \mid \Gamma, x : A$

Beyond the standard typing derivations of the simply typed lambda calculus, there is a new *command* typing judgement with two contexts:

$\Gamma; \Delta \vdash t ! A$

The additional command terms are given by an arrow abstraction and application, and a unit and bind for arrow types.

$$\frac{\Gamma; x : A \vdash t ! B}{\Gamma \vdash \lambda^\bullet x. t : A \multimap B}$$

$$\frac{\Gamma \vdash u : A \multimap B \quad \Gamma, \Delta \vdash v : A}{\Gamma; \Delta \vdash u \bullet v ! B}$$

$$\frac{\Gamma, \Delta \vdash u : A}{\Gamma; \Delta \vdash \text{return } u ! A}$$

$$\frac{\Gamma; \Delta \vdash t ! A \quad \Gamma; \Delta, x : A \vdash s ! B}{\Gamma; \Delta \vdash \text{do } x \leftarrow t \text{ in } s ! B}$$

Note the adjusted syntax for the return and bind terms for consistency with the languages of this paper. There are equations for beta and eta equality of the arrow abstraction, and the monad laws.

$$\frac{\Gamma; x : A \vdash t ! B \quad \Gamma, \Delta \vdash u : A}{\Gamma; \Delta \vdash \lambda^\bullet x. t \bullet u = t[u/x] ! B}$$

$$\frac{\Gamma \vdash u : A \multimap B}{\Gamma \vdash \lambda^\bullet x. (u \bullet x) = u : A \multimap B}$$

$$\frac{\Gamma, \Delta \vdash u : A \quad \Gamma; \Delta, x : A \vdash t ! B}{\Gamma; \Delta \vdash \text{do } x \leftarrow \text{return } u \text{ in } t = t[u/x] ! B}$$

$$\frac{\Gamma; \Delta \vdash t ! B}{\Gamma; \Delta \vdash \text{do } x \leftarrow t \text{ in return } x = t ! B}$$

$$\frac{\Gamma; \Delta \vdash r ! A \quad \Gamma; \Delta, x : A \vdash s ! B \quad \Gamma; \Delta, y : B \vdash t ! C}{\Gamma; \Delta \vdash \text{do } x \leftarrow r \text{ in } (\text{do } y \leftarrow s \text{ in } t) = \text{do } y \leftarrow (\text{do } x \leftarrow r \text{ in } s) \text{ in } t ! C}$$

Motivated by the similarities to the RMM, we provide a sound and complete denotational semantics for the arrow calculus using a refined notion of strong relative monads (Section 4.2).

Definition 6.1. An arrow calculus model consists of

- A Cartesian closed category $\mathcal{A}$
- A W -strong J -relative monad $(T, \eta, (-)^*)$

where J and W are defined as follows. If $\mathcal{A}$ is a Cartesian closed category, we write $\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ for the category of $\mathcal{A}$ -enriched functors $\mathcal{A}^{\text{op}} \rightarrow \mathcal{A}$ and enriched natural transformations [33]. Then $J, K : \mathcal{A} \rightarrow \mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ are defined by the *enriched Yoneda embedding*, $JAB = A^B$, and the constant preseaf, $KAB = A$, respectively. Finally, $W : \mathcal{A} \times \mathcal{A} \rightarrow \mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ is defined as $W(A, B) = KA \times JB$ and intuitively captures the two contexts in the arrow calculus command judgements.

We interpret judgements $\Gamma \vdash u : A$ as morphisms in $\mathcal{A}$ and the terms of the simply typed lambda calculus by their standard interpretation in the Cartesian closed category $\mathcal{A}$ [26, 38]. The new type is interpreted as

$$\llbracket A \leadsto B \rrbracket = T\llbracket B \rrbracket\llbracket A \rrbracket, \quad (6)$$

and the new typing judgement is interpreted as a morphism in $\mathcal{A}$.

$$\llbracket \Gamma; \Delta \vdash s ! B \rrbracket : \llbracket \Gamma \rrbracket \rightarrow T\llbracket B \rrbracket\llbracket \Delta \rrbracket$$

The interpretation of the remaining terms is defined inductively.

$$\llbracket \Gamma \vdash \lambda^* x. t : A \leadsto B \rrbracket = \llbracket \Gamma; x : A \vdash t ! B \rrbracket$$

$$\begin{aligned} \llbracket \Gamma; \Delta \vdash u \bullet v ! B \rrbracket &= \text{ev}_{T\llbracket B \rrbracket\llbracket \Delta \rrbracket, T\llbracket B \rrbracket\llbracket A \rrbracket} \circ (T\llbracket B \rrbracket\llbracket A \rrbracket, \llbracket \Delta \rrbracket \circ \text{curry}_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, \llbracket A \rrbracket}(\llbracket \Gamma, \Delta \vdash v : A \rrbracket)), \\ &\llbracket \Gamma \vdash u : A \leadsto B \rrbracket) \end{aligned}$$

$$\llbracket \Gamma; \Delta \vdash \text{return } u ! A \rrbracket = \chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, T\llbracket A \rrbracket}(\eta_A \circ \chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, J\llbracket A \rrbracket}^{-1}(\text{curry}_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, \llbracket A \rrbracket}(\llbracket \Gamma, \Delta \vdash u : A \rrbracket)))$$

$$\begin{aligned} \llbracket \Gamma; \Delta \vdash \text{do } x \leftarrow t \text{ in } s ! B \rrbracket &= \chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, T\llbracket B \rrbracket} \left(\left(\chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket \times \llbracket A \rrbracket, T\llbracket B \rrbracket}^{-1}(\llbracket \Gamma; \Delta, x : A \vdash s ! B \rrbracket) \right) \right. \\ &\quad \left. \circ (\text{id}_{K\llbracket \Gamma \rrbracket} \times \kappa_{\llbracket \Delta \rrbracket, \llbracket A \rrbracket})^* \circ (\text{id}_{K\llbracket \Gamma \rrbracket \times J\llbracket \Delta \rrbracket}, \chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, T\llbracket A \rrbracket}^{-1}(\llbracket \Gamma; \Delta \vdash t ! A \rrbracket)) \right) \end{aligned}$$

Here, $\text{ev}_{A,B} : A^B \times B \rightarrow A$ is the evaluation morphism given by the closed structure, $\text{curry}_{A,B,C} : \mathcal{A}(A \times B, C) \rightarrow \mathcal{A}(A, C^B)$ is the closed structure adjunction, $TA_{B,C} : B^C \rightarrow TAC^{TAB}$ is the enriched functor components, $\kappa_{A,B} : JA \times JB \rightarrow J(A \times B)$ is the canonical map, and $\chi_{A,B,F} : \mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})(KA \times JB, F) \xrightarrow{\sim} \mathcal{A}(A, FB)$ is the adjunction given by $\theta \mapsto \lambda a. \theta_B(a, \lambda b. b)$ in the forward direction and $f \mapsto \theta$, where $\theta_C = \text{ev}_{FC, FB} \circ (F_{B,C} \circ \pi_{B^C}, f \circ \pi_A) : A \times B^C \rightarrow FC$, in the reverse direction.

THEOREM 6.2. *The models of the arrow calculus are a sound and complete semantics.*

PROOF. Soundness is deduced from the following substitution lemmas, which are proved by structural induction.

$$\llbracket \Gamma \vdash u[v/b] : B \rrbracket = \llbracket \Gamma, a : A \vdash u : B \rrbracket \circ (\text{id}_{\llbracket \Gamma \rrbracket}, \llbracket \Gamma \vdash v : A \rrbracket)$$

$$\llbracket \Gamma; \Delta \vdash t[u/a] ! B \rrbracket = \llbracket \Gamma, a : A; \Delta \vdash t ! B \rrbracket \circ (\text{id}_{\llbracket \Gamma \rrbracket}, \llbracket \Gamma \vdash u : A \rrbracket)$$

$$\begin{aligned} \llbracket \Gamma; \Delta \vdash t[u/a] ! B \rrbracket &= \chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, T\llbracket B \rrbracket} \left(\chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket \times \llbracket A \rrbracket, T\llbracket B \rrbracket}^{-1}(\llbracket \Gamma; \Delta, a : A \vdash t ! B \rrbracket) \right. \\ &\quad \left. \circ (\pi_{K\llbracket \Gamma \rrbracket}, \kappa_{\llbracket \Delta \rrbracket, \llbracket A \rrbracket} \circ (\pi_{J\llbracket \Delta \rrbracket}, \chi_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, J\llbracket A \rrbracket}^{-1}(\text{curry}_{\llbracket \Gamma \rrbracket, \llbracket \Delta \rrbracket, \llbracket A \rrbracket}(\llbracket \Gamma, \Delta \vdash u : A \rrbracket)))) \right) \end{aligned}$$

Completeness is proved via the term model construction. The simply typed lambda calculus terms give the Cartesian closed structure [26, 38]. The object mapping for the W -strong J -relative monad is $(- \rightsquigarrow B)$, where the enriched functorial action is given by $\lambda^{\bullet} a. t \bullet f a$ for $f : A' \rightarrow A$ and $t : A \rightsquigarrow B$. Next, every equivalence class of terms $[\Gamma; \Delta \vdash t ! B]$ corresponds under the adjunction χ to a natural map $K[\Gamma] \times J[\Delta] \rightarrow (- \rightsquigarrow [B])$. The equivalence classes of return t and do $x \leftarrow p$ in q terms in contexts $\Gamma; \Delta$ under the adjunction then define the W -strong J -relative monad structure. $\square$

6.1 The arrow relative monadic metalanguage (ARMM)

Identifying the models of the arrow calculus as a W -strong relative monad suggests that it can be extended to the framework of RMM. We begin by identifying the key features of the models in Definition 6.1 to achieve this. The command judgement (6), while interpreted as a morphism in $\mathcal{A}$, is primarily used in the semantics as a morphism in $\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$. The critical link is via the following isomorphism:

$$\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})(KA \times JB, TC) \cong \mathcal{A}(A, TCB). \quad (7)$$

The objects in $\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ can be thought of as arrow types with parametric input. In Definition 6.1, the semantics of the types JA is given by the enriched Yoneda embedding, and so can be understood as pure arrow types with output A . Types KA have semantics via constant presheaves and do not interact with the relative monad structure. The role of these types is via (7); they permit the movement of variables between $\mathcal{A}$ and $\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ contexts. Finally, types TA are effectful arrow types with output A , parametric over some input.

However, the role of $\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ is not essential apart from providing canonical interpretations of J and K . We can therefore abstract the explicit category $\mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ to an arbitrary Cartesian category $\mathcal{C}$, while maintaining Cartesian functors $J, K : \mathcal{A} \rightarrow \mathcal{C}$ satisfying certain conditions. In particular, we require that parametric arrow types in $\mathcal{C}$ can be abstracted over a pure input (capturing the isomorphism (7)), and abstracting pure arrows results in function types:

$$\mathcal{C}(KA \times JB, X) \cong \mathcal{A}(A, B \Rightarrow X) \quad (8)$$

$$\mathcal{A}(A \times B, C) \cong \mathcal{A}(A, B \Rightarrow JC) \quad (9)$$

Then, we can define the original arrow types as $A \rightsquigarrow B ::= A \Rightarrow TB$ and function types as $A \rightarrow B ::= A \Rightarrow JB$. We additionally require a natural coercion from K to J to allow variables in the J context to be moved to variables in the K context. This is admissible in the arrow calculus and is semantically validated by the canonical map from the Yoneda embedding to the constant presheaf functor. It is derivable from (8) and (9) by applying the following isomorphism to the identity morphism.

$$\mathcal{A}(A, A) \cong \mathcal{A}(A \times 1, A) \cong \mathcal{A}(A, 1 \Rightarrow JA) \cong \mathcal{C}(KA \times J1, JA) \cong \mathcal{C}(KA, JA)$$

We thus define the arrow relative monadic metalanguage (ARMM) as follows.

$$\begin{aligned} \mathcal{A} \text{ types:} \quad & A, B ::= 1 \mid A \times B \mid A \Rightarrow X \\ \mathcal{C} \text{ types:} \quad & X, Y ::= 1 \mid X \times Y \mid JA \mid KA \mid TA \\ \mathcal{A} \text{ contexts:} \quad & \Gamma, \Delta ::= \emptyset \mid \Gamma, a : A \\ \mathcal{C} \text{ contexts:} \quad & \Phi ::= \emptyset \mid \Phi, x : X \end{aligned}$$

The ARMM terms for $\mathcal{A}$ -types are the standard terms for a Cartesian category. The ARMM terms for $\mathcal{C}$ -types beyond the standard Cartesian ones are:

$$\frac{\Gamma; \Delta; \Phi \vdash_C t : KA \quad \Gamma, a : A; \Delta; \Phi \vdash_C s : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } t \text{ in } s : X} \quad \frac{\Gamma; \Delta; \Phi \vdash_C t : JA \quad \Gamma; \Delta, a : A; \Phi \vdash_C s : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } t \text{ in } s : X}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C t : JA}{\Gamma; \Delta; \Phi \vdash_C \text{return } t : TA} \quad \frac{\Gamma; \Delta; \Phi \vdash_C t : TA \quad \Gamma; \Delta; x : JA \vdash_C s : TB}{\Gamma; \Delta; \Phi \vdash_C \text{do } x \leftarrow t \text{ in } s : TB}$$

Note that $\mathcal{C}$ judgements have three contexts. The first two typing rules validate that context Γ is a K context and context Δ is a J context, similar to the style of the arrow calculus. The third context Φ can have arbitrary $\mathcal{C}$ types, such as computations, and permits categorical semantics via the general RMM framework. The latter two typing rules are the standard J -relative monad terms for T , where binding is restricted to K and J contexts. The ARMM terms for the interaction between $\mathcal{A}$ -types and $\mathcal{C}$ -types are:

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A \Rightarrow X \quad \Gamma, \Delta \vdash_{\mathcal{A}} v : A}{\Gamma; \Delta; \Phi \vdash_C u \bullet v : X} \quad \frac{\Gamma; a : A; \emptyset \vdash_C t : X}{\Gamma \vdash_{\mathcal{A}} \lambda(a : A).t : A \Rightarrow X}$$

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A}{\Gamma; \Delta; \Phi \vdash_C K(u) : KA} \quad \frac{\Gamma, \Delta \vdash_{\mathcal{A}} u : A}{\Gamma; \Delta; \Phi \vdash_C J(u) : JA} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \Rightarrow JB \quad \Gamma \vdash_{\mathcal{A}} v : A}{\Gamma \vdash_{\mathcal{A}} uv : B}$$

The full term calculus and equational theory of ARMM is given in Appendix B.4. Regular application and abstraction under the derivable function type $A \rightarrow B ::= A \Rightarrow JB$ are:

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A \rightarrow B \quad \Gamma \vdash_{\mathcal{A}} v : A}{\Gamma \vdash_{\mathcal{A}} uv : B} \quad \frac{\Gamma, a : A \vdash_{\mathcal{A}} u : B}{\Gamma \vdash_{\mathcal{A}} \lambda(a : A).J(u) : A \rightarrow B}$$

Definition 6.3. A model of ARMM consists of

- Cartesian categories $\mathcal{A}$ and $\mathcal{C}$
- Cartesian functors $J, K : \mathcal{A} \rightarrow \mathcal{C}$
- A W -strong J -relative monad $(T, \eta, (-)^*)$, where $W(A, B) = KA \times JB$.
- An adjunction $(K - \times JA) \dashv (A \Rightarrow -)$ for each $A \in |\mathcal{A}|$.
- An adjunction $(- \times A) \dashv (A \Rightarrow J-)$ for each $A \in |\mathcal{A}|$.

The $\mathcal{A}$ judgements are interpreted by morphisms in $\mathcal{A}$ as usual, while the $\mathcal{C}$ judgements are given by morphisms in $\mathcal{C}$:

$$\llbracket \Gamma; \Delta; \Phi \vdash_C t : X \rrbracket : K[\llbracket \Gamma \rrbracket] \times J[\llbracket \Delta \rrbracket] \times [\llbracket \Phi \rrbracket] \rightarrow [\llbracket X \rrbracket]$$

The non-standard interaction terms are interpreted inductively as:

$$\begin{aligned} \llbracket \Gamma; \Delta; \Phi \vdash_C u \bullet v : X \rrbracket &= \chi_{[\llbracket \Gamma \rrbracket], [\llbracket A \rrbracket], [\llbracket X \rrbracket]}^{-1} (\llbracket \Gamma \vdash_{\mathcal{A}} u : A \Rightarrow X \rrbracket) \\ &\quad \circ (\pi_{K[\llbracket \Gamma \rrbracket]}, J[\llbracket \Gamma, \Delta \vdash_{\mathcal{A}} v : A \rrbracket] \circ \kappa_{[\llbracket \Gamma \rrbracket], [\llbracket A \rrbracket]}^J \circ (\psi_{[\llbracket \Gamma \rrbracket]} \circ \pi_{K[\llbracket \Gamma \rrbracket]}, \pi_{J[\llbracket A \rrbracket]})) \\ \llbracket \Gamma \vdash_{\mathcal{A}} \lambda(x : A).t : A \Rightarrow X \rrbracket &= \chi_{[\llbracket \Gamma \rrbracket], [\llbracket A \rrbracket], [\llbracket X \rrbracket]} (\llbracket \Gamma; x : A; \emptyset \vdash_C t : X \rrbracket) \\ \llbracket \Gamma; \Delta; \Phi \vdash_C K(u) : KA \rrbracket &= K[\llbracket \Gamma \vdash_{\mathcal{A}} u : A \rrbracket] \circ \pi_{K[\llbracket \Gamma \rrbracket]} \\ \llbracket \Gamma; \Delta; \Phi \vdash_C J(u) : JA \rrbracket &= J[\llbracket \Gamma, \Delta \vdash_{\mathcal{A}} u : A \rrbracket] \circ \kappa_{[\llbracket \Gamma \rrbracket], [\llbracket A \rrbracket]}^J \circ (\psi_{[\llbracket \Gamma \rrbracket]} \circ \pi_{K[\llbracket \Gamma \rrbracket]}, \pi_{J[\llbracket A \rrbracket]}) \\ \llbracket \Gamma \vdash_{\mathcal{A}} uv : B \rrbracket &= \text{curry}_{[\llbracket \Gamma \rrbracket], [\llbracket A \rrbracket], [\llbracket B \rrbracket]}^{-1} (\llbracket \Gamma \vdash_{\mathcal{A}} u : A \Rightarrow JB \rrbracket) \circ (\text{id}_{[\llbracket \Gamma \rrbracket]}, \llbracket \Gamma \vdash_{\mathcal{A}} v : A \rrbracket) \end{aligned}$$

where $\chi_{A,B,X} : \mathcal{C}(KA \times JB, X) \rightarrow \mathcal{A}(A, B \Rightarrow X)$ and $\text{curry}_{A,B,C} : \mathcal{A}(A \times B, C) \rightarrow \mathcal{A}(A, B \Rightarrow JC)$ are the components of the adjunctions, respectively, $\psi_A : KA \rightarrow JA$ are the components of the

natural coercion, and $\kappa^J : JA \times JB \rightarrow J(A \times B)$ and $\kappa^K : KA \times KB \rightarrow K(A \times B)$ are the canonical isomorphisms.

Example 6.4. If $\mathcal{A}$ is a Cartesian closed category and $T : \mathcal{A} \rightarrow \mathcal{ACat}(\mathcal{A}^{\text{op}}, \mathcal{A})$ is a W -strong J -relative monad, then $(\mathcal{A}, J, K, T)$ is a model of ARMM where J and K are as in Definition 6.1.

THEOREM 6.5. *The models of ARMM are a sound semantics.*

PROOF. Soundness is shown via structural induction on the following substitution lemmas.

$$\begin{aligned} \llbracket \Gamma \vdash u[v/b] : B \rrbracket &= \llbracket \Gamma, a : A \vdash u : B \rrbracket \circ (\text{id}_{\llbracket \Gamma \rrbracket}, \llbracket \Gamma \vdash v : A \rrbracket) \\ \llbracket \Gamma; \Delta; \Phi \vdash_C t[u/a] : X \rrbracket &= \llbracket \Gamma, a : A; \Delta; \Phi \vdash_C t : X \rrbracket \\ &\quad \circ (\kappa_{\llbracket \Gamma \rrbracket, \llbracket A \rrbracket}^K \circ (\pi_{K[\llbracket \Gamma \rrbracket]}, K[\llbracket \Gamma \vdash_{\mathcal{A}} u : A \rrbracket \circ \pi_{K[\llbracket \Gamma \rrbracket]}], \pi_{J[\llbracket \Delta \rrbracket]}, \pi_{\llbracket \Phi \rrbracket})) \\ \llbracket \Gamma; \Delta; \Phi \vdash_C t[u/a] : X \rrbracket &= \llbracket \Gamma; \Delta, a : A; \Phi \vdash_C t : X \rrbracket \circ (\pi_{K[\llbracket \Gamma \rrbracket]}, \\ &\quad \kappa_{\llbracket \Delta \rrbracket, \llbracket A \rrbracket}^J \circ (\pi_{J[\llbracket \Delta \rrbracket]}, J[\llbracket \Gamma, \Delta \vdash_{\mathcal{A}} u : A \rrbracket \circ (\psi_{\llbracket \Gamma \rrbracket} \circ \pi_{K[\llbracket \Gamma \rrbracket]}, \pi_{J[\llbracket \Delta \rrbracket]})]), \pi_{\llbracket \Phi \rrbracket})) \\ \llbracket \Gamma; \Delta; \Phi \vdash_C t[s/y] : X \rrbracket &= \llbracket \Gamma; \Delta; \Phi \vdash_C t[s/y] : X \rrbracket \circ (\text{id}_{K[\llbracket \Gamma \rrbracket] \times J[\llbracket \Delta \rrbracket] \times \llbracket \Phi \rrbracket]}, \llbracket \Gamma; \Delta; \Phi \vdash_C s : Y \rrbracket) \end{aligned}$$

The equations in Appendix B.4 are also validated by structural induction. The adjunction χ gives the β and η laws for application and abstraction of arrows. $\square$

With this result, we can show that our calculus subsumes the arrow calculus. The syntactic interpretation of their types is $[A \rightarrow B] = (A \Rightarrow JB)$ and $[A \rightsquigarrow B] = (A \Rightarrow TB)$. The command judgement (6) is interpreted as

$$[\Gamma; \Delta \vdash t ! A] = \Gamma; \Delta; \emptyset \vdash_C t : TA$$

while their terms are interpreted as:

$$\begin{aligned} [\Gamma \vdash \lambda^{\bullet} x. t : A \rightsquigarrow B] &= \Gamma \vdash_{\mathcal{A}} \lambda(x : A). t : A \Rightarrow TB \\ [\Gamma; \Delta \vdash u \bullet v ! B] &= \Gamma; \Delta; \emptyset \vdash_C u \bullet v : TB \\ [\Gamma; \Delta \vdash \text{return } u ! A] &= \Gamma; \Delta; \emptyset \vdash_C \text{return } J(u) : TA \\ [\Gamma; \Delta \vdash \text{do } x \leftarrow t \text{ in } s ! B] &= \Gamma; \Delta; \emptyset \vdash_C \text{do } y \leftarrow t \text{ in } (\text{let } J(x) \text{ be } y \text{ in } s) : TB \end{aligned}$$

THEOREM 6.6. *The interpretation of the arrow calculus in ARMM is a conservative extension.*

PROOF. From Example 6.4, all models of the arrow calculus $(\mathcal{A}, T)$ are models of ARMM. Furthermore, the semantics of the interpretation of an arrow calculus term coincides, or corresponds via χ , with its semantics as a model of the arrow calculus. Hence, the term model must validate all the equations of ARMM. $\square$

Remark. The ARMM calculus could be generalised further by allowing binds in arbitrary contexts.

$$\frac{\Gamma; \Delta; \Phi \vdash_C t : TA \quad \Gamma; \Delta; \Phi, x : JA \vdash_C s : TB}{\Gamma; \Delta; \Phi \vdash_C \text{do } x \leftarrow t \text{ in } s : TB}$$

This would make the models consist of a strong J -relative monad, rather than a W -strong J relative monad. We cannot apply Proposition 4.14 to equate these notions of strength because $\mathcal{A}$ is not cocomplete. However, an analogous conservative extension result may be possible by embedding $\mathcal{A}$ into a cocomplete category, such as $\widehat{\mathcal{A}}$.

7 Conclusion and future work

We have introduced a language RMM for strong relative monads and demonstrated a significant advantage to working with term calculi based on it. RMM generalises the monadic metalanguage [50] in two directions:

- It allows us to *restrict* our notion of computation by programming with C types that only allow computations on another sort of types $\mathcal{A}$. This was done in Section 3.2 to restrict to computations on *finite* datatypes and thus provide a more fine-grained approach to computation.
- It allows us to *expand* our notion of computation. By programming with $\mathcal{A}$ types with *partial access* to an additional sort C , it permits a broader notion of computations than what is allowed in $\mathcal{A}$. We saw in Section 5 and Section 6 that this allowed programming with cost-sensitive computations through graded monads and a wider variety of effects through arrows.

In particular, the languages LNL-RMM introduced in Section 5 for graded monads and ARMM in Section 6 for arrows:

- (1) Are no less general than their predecessors since we prove conservative extension theorems, Theorem 5.13 and Theorem 6.6.
- (2) Are more expressive than their predecessors, where, in particular, LNL-RMM has grade types in the language, which allows for more flexible programming.
- (3) Come with built-in sound denotational semantics from the RMM, as shown in Theorem 5.12 and Theorem 6.5.

The denotational semantics of all the example languages in the paper, including the new semantics provided for the arrow calculus (Theorem 6.2), was possible due to the notion of strong relative monad we introduced in Section 3, and its generalisations in Section 4.2. We justified that this definition of strong relative monads is the appropriate one for semantics of programming languages by establishing its equivalence to enriched relative monads (Theorem 4.6). We also extended the broader literature on relative monads [2–4, 65] to facilitate the design of these languages.

Future work. We have focused on two applications of relative monads: graded monads and arrows. The positive results suggest that other applications of relative monads will also benefit from taking the relative monadic metalanguage as a starting point. One example is the *simple relational specification monads* [45], which are (strong) relative monads on the product functor $\text{Set} \times \text{Set} \rightarrow \text{Set}$.

Since we have shown that the relative monadic metalanguage is a unifying starting point for diverse aspects of programming languages, we envisage that more advanced methods from monad theory will benefit from a general analysis in terms of relative monads. We have in mind methods from program logics, such as monad liftings for logical relations [29, 32], which have not yet been investigated from the relative point of view.

Acknowledgements. This work was funded in part by ERC Grant BLAST, AFOSR under award number FA9550-21-1-0038, grants from ARIA Safeguarded AI, and the Clarendon Scholarship.

We thank the anonymous reviewers and many people for helpful discussions and suggestions, including Dylan McDermott, Nathanael Arkor and Nathan Corbyn.

References

- [1] Thosten Altenkirch, James Chapman, and Tarmo Uustalu. 2010. Monads need not be endofunctors. In *Proc. FoSSaCS '10*. doi:10.2168/LMCS-11(1:3)2015

- [2] Nathanael Arkor and Dylan McDermott. 2024. The formal theory of relative monads. *J. Pure Appl. Algebra* (2024). doi:10.1016/j.jpaa.2024.107676
- [3] Nathanael Arkor and Dylan McDermott. 2025. The nerve theorem for relative monads. *TAC* (2025). doi:10.70930/tac/jjc9mnlky
- [4] Nathanael Arkor and Dylan McDermott. 2025. Relative monadicity. *J. Algebra* (2025). doi:10.1016/j.jalgebra.2024.08.040
- [5] Robert Atkey. 2011. What is a Categorical Model of Arrows?. In *Proc. MSFP '08*. doi:10.1016/j.entcs.2011.02.014
- [6] Michael Barr and Charles Wells. 1985. *Toposes, Triples and Theories*. Springer, New York.
- [7] Nick Benton. 1994. A Mixed Linear and Non-Linear Logic: Proofs, Terms and Models (Extended Abstract). In *Proc. CSL '94*. doi:10.5555/647844.736565
- [8] Nick Benton, Gavin Bierman, Valeria de Paiva, and Martin Hyland. 1993. A term calculus for Intuitionistic Linear Logic. In *Proc. TLCA '93*. doi:10.1007/BFb0037099
- [9] Clemens Berger, Paul-André Melliès, and Mark Weber. 2012. Monads with arities and their associated theories. *J. Pure Appl. Algebra* (2012). doi:10.1016/j.jpaa.2012.02.039
- [10] Francis Borceux. 1994. *Handbook of categorical algebra: Basic category theory*. Vol. 1. Cambridge University Press.
- [11] John Bourke and Richard Garner. 2019. Monads and theories. *Adv. Math.* (2019). doi:10.1016/j.aim.2019.05.016
- [12] Ezgi Çiçek, Marco Gaboardi, and Deepak Garg. 2016. Cost-analysis: How do monads and comonads differ? Slides available from the author's webpage. <https://drezgi.wordpress.com/wp-content/uploads/2019/01/dice16.pdf>
- [13] Antony Courtney and Conal Elliott. 2001. Genuinely functional user interfaces. In *Haskell Workshop*. 41–69.
- [14] Brian Day. 1970. On closed categories of functors. In *Reports of the Midwest Category Seminar IV*. doi:10.1007/BFb0060438
- [15] Jeff Egger, Rasmus Ejlers Møgelberg, and Alex Simpson. 2009. Enriching an Effect Calculus with Linear Types. In *Proc. CSL 2009*. doi:10.1007/978-3-642-04027-6_19
- [16] Tobias Fritz. 2020. A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Adv. Math.* (2020). doi:10.1016/j.aim.2020.107239
- [17] Tobias Fritz, Tomáš Gonda, Nicholas Houghton-Larsen, Antonio Lorenzin, Paolo Perrone, and Dario Stein. 2023. Dilations and information flow axioms in categorical probability. *Math. Struct. Comput. Sci.* (2023). doi:10.1017/S0960129523000324
- [18] Soichiro Fujii, Shinya Katsumata, and Paul-André Melliès. 2016. Towards a Formal Theory of Graded Monads. In *Proc. FoSSaCS '16*. doi:10.1007/978-3-662-49630-5_30
- [19] Jean-Yves Girard. 1987. Linear logic. *Theor. Comput. Sci.* (1987). doi:10.1016/0304-3975(87)90045-4
- [20] Michèle Giry. 1982. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*. doi:10.1007/BFb0092872
- [21] Claudio Hermida. 2000. Representable Multicategories. *Adv. Math.* (2000). doi:10.1006/aima.1999.1877
- [22] Michael Hicks, Gavin M. Bierman, Nataliya Guts, Daan Leijen, and Nikhil Swamy. 2014. Polymonadic Programming. In *Proc. MSFP '14*. doi:10.4204/EPTCS.153.7
- [23] Steven Holtzen, Guy Van den Broeck, and Todd Millstein. 2020. Scaling exact inference for discrete probabilistic programs. In *Proc. OOPSLA '20*. doi:10.1145/3428208
- [24] Paul Hudak, Antony Courtney, Henrik Nilsson, and John Peterson. 2003. Arrows, Robots, and Functional Reactive Programming. In *Proc. AFP '02*. doi:10.1007/978-3-540-44833-4_6
- [25] John Hughes. 2000. Generalising monads to arrows. *Sci. Comput. Program.* (2000). doi:10.1016/S0167-6423(99)00023-4
- [26] Bart Jacobs. 2001. *Categorical logic and type theory*. Elsevier. doi:10.2307/421214
- [27] Patrik Jansson and Johan Jeuring. 1999. Polytropic Compact Printing and Parsing. In *Proc. ESOP '99*. doi:10.1007/3-540-49099-X_18
- [28] Yuchen Jiang, Runze Xue, and Max S. New. 2025. Notions of Stack-Manipulating Computation and Relative Monads. *Proc. OOPSLA '25* (2025). doi:10.1145/3720434
- [29] Shinya Katsumata. 2005. A semantic formulation of TT-lifting and logical predicates for computational metalanguage. In *Proc. CSL '05*. doi:10.1007/11538363_8
- [30] Shinya Katsumata. 2014. Parametric effect monads and semantics of effect systems. In *Proc. POPL '14*. doi:10.1145/2535838.2535846
- [31] Shinya Katsumata, Dylan McDermott, Tarmo Uustalu, and Nicolas Wu. 2022. Flexible presentations of graded monads. In *Proc. ICFP '22*. doi:10.1145/3547654
- [32] Shinya Katsumata, Tetsuya Sato, and Uustalu Tarmo. 2018. Codensity Lifting of Monads and its Dual. *LMCS* (2018). doi:10.23638/LMCS-14(4:6)2018
- [33] Max Kelly. 1982. *Basic Concepts of Enriched Category Theory*. Cambridge University Press.
- [34] Max Kelly and John Power. 1993. Adjunctions whose counits are coequalizers, and presentations of finitary enriched monads. *J. Pure Appl. Algebra* (1993). doi:10.1016/0022-4049(93)90092-8
- [35] Anders Kock. 1970. Monads on symmetric monoidal closed categories. *Archiv der Mathematik* (1970). doi:10.1007/BF01220868

- [36] Anders Kock. 1972. Strong functors and monoidal monads. *Archiv der Mathematik* (1972). doi:10.1007/BF01304852
- [37] Satoshi Kura. 2020. Graded Algebraic Theories. In *Proc. FoSSaCS '20*. doi:10.1007/978-3-030-45231-5_2
- [38] Joachim Lambek and Philip J. Scott. 1986. *Introduction to higher order categorical logic*. Cambridge University Press.
- [39] Paul Blain Levy. 2006. *Call by push value: a subsuming paradigm*. Springer. doi:10.1007/3-540-48959-2_17
- [40] Paul Blain Levy, John Power, and Hayo Thielecke. 2003. Modelling environments in call-by-value programming languages. *Inform. Comput.* (2003). doi:10.1016/S0890-5401(03)00088-9
- [41] Alexander Lew, Marco Cusumano-Towner, Benjamin Sherman, Michael Carbin, and Vikash Mansinghka. 2020. Trace types and denotational semantics for sound programmable inference in probabilistic languages. In *Proc. POPL '20*. doi:10.1145/3371087
- [42] Jack Liell-Cock and Samuel Staton. 2025. Compositional imprecise probability: a solution from graded monads and Markov categories. In *Proc. POPL '25*. doi:10.1145/3704890
- [43] Sam Lindley, Philip Wadler, and Jeremy Yallop. 2010. The arrow calculus. *J. Funct. Program.* (2010). doi:10.1017/S095679680999027X
- [44] Saunders Mac Lane. 1998. *Categories for the Working Mathematician*. Springer, New York.
- [45] Kenji Maillard, Cătălin Hrițcu, Exequiel Rivas, and Antoine Van Muylder. 2020. The next 700 relational program logics. In *Proc. POPL '20*. doi:10.1145/3371072
- [46] Ernest G. Manes. 2012. *Algebraic theories*. Springer, New York. doi:10.1007/978-1-4612-9860-1
- [47] Francisco Marmolejo and Richard Wood. 2010. Monads as Extension Systems - No Iteration is Necessary. *TAC* (2010). doi:10.70930/tac/sbn7nctd
- [48] Dylan McDermott and Tarmo Uustalu. 2022. Flexibly graded monads and graded algebras. In *Proc. MPC '22*. doi:10.1007/978-3-031-16912-0_4
- [49] Dylan McDermott and Tarmo Uustalu. 2022. What makes a strong monad?. In *Proc. MSFP '22*. doi:10.4204/EPTCS.360.6
- [50] Eugenio Moggi. 1991. Notions of computation and monads. *Inf. Comput.* (1991). doi:10.1016/0890-5401(91)90052-4
- [51] Dominic Orchard, Philip Wadler, and Harley Eades III. 2020. Unifying graded and parameterised monads. In *Proc. MSFP '20*. doi:10.4204/EPTCS.317.2
- [52] Ross Paterson. 2001. A New Notation for Arrows. In *Proc. ICFP '01*. doi:10.1145/507669.507664
- [53] Michael Pilquist, Runar Bjarnason, and Paul Chiusano. 2014. *Functional Programming in Scala*. Manning. doi:10.5555/2688794
- [54] Andrew Pitts. 2013. *Nominal Sets: names and symmetry in computer science*. Cambridge University Press. doi:10.5555/2512979
- [55] Gordon Plotkin and John Power. 2002. Notions of Computation Determine Monads. In *Proc. FoSSaCS '02*. doi:10.1007/3-540-45931-6_24
- [56] Gordon Plotkin and John Power. 2003. Algebraic operations and generic effects. *Appl. Categ. Struct.* (2003). doi:10.1023/A:1023064908962
- [57] Mario Román. 2022. Promonads and String Diagrams for Effectful Categories. In *Proc. ACT '22*. doi:10.4204/EPTCS.380.20
- [58] Takahiro Sanada. 2024. Algebraic effects and handlers for arrows. *JFP* (2024). doi:10.1017/S0956796824000066
- [59] Andrew Slattery. [n. d.]. Strong relative monads on multicategories and on monoidal categories. PDF available on author's webpage. https://aslaterry.co.uk/files/Strong_Relative_Monads_on_Multicategories_and_on_Monoidal_Categories.pdf
- [60] Andrew Slattery. 2024. *Commutativity of Relative Pseudomonads*. Ph.D. Dissertation. University of Leeds.
- [61] Sam Staton. 2010. Completeness for Algebraic Theories of Local State. In *Proc. FoSSaCS '10*. doi:10.1007/978-3-642-12032-9_5
- [62] Sam Staton. 2014. Freyd categories are Enriched Lawvere Theories. In *Proc. WACT '13*. doi:10.1016/j.entcs.2014.02.010
- [63] Marshall Stone. 1949. Postulates for the barycentric calculus. *Annali di Matematica Pura ed Applicata* (1949). doi:10.1007/bf02413910
- [64] Ross Tate. 2013. The sequential semantics of producer effect systems. In *Proc. POPL '13*. doi:10.1145/2429069.2429074
- [65] Tarmo Uustalu. 2010. Strong Relative Monads (Extended Abstract). In *Proc. CMCS '10*. <https://cs.ioc.ee/~tarmo/papers/uustalu-cmcs10short.pdf>
- [66] Philip Wadler. 1993. Monads for functional programming. In *Proc. Program Design Calculi '93*. doi:10.1007/978-3-662-02880-3_8

A Bistrong relative monads on non-symmetric monoidal categories

If $\mathcal{C}$ is a non-symmetric monoidal category, Definition 3.3 no longer allows sequencing of computations inside arbitrary contexts. It permits a bind for terms with contexts of the form $\Gamma, x : JA$, but not the generic form of $\Gamma, x : JA, \Gamma'$. In this more general setting, the notion described in

Definition 3.3 is referred to as a *left strong monad*. A *right strong monad* can be defined analogously, and formally understood as a left strong monad on the *reverse monoidal category*.

Definition A.1. If C is a monoidal category, its reverse C^{rev} is defined to have the same underlying category, but $X \otimes^{\text{rev}} Y = Y \otimes X$, $\rho^{\text{rev}} = \lambda$, $\lambda^{\text{rev}} = \rho$ and $\alpha^{\text{rev}} = \alpha^{-1}$

In the non-relative case, when C is not a symmetric monoidal category, it is common to consider a *bistrong monad* on C . Usually, a bistrength for a monad is presented as a compatible left and right strength [49]. However, as we have seen, strength maps are insufficient for relative monads. The Kleisli form of a bistrong relative monad provides the more suitable notion. The following is an instantiation of a *multicategorical strong relative monad* [60] for a representable multicategory [21]:

Definition A.2. A *bistrong J -relative monad* $(T, \eta, (-)^*)$, consists of:

- An object $TA \in |C|$ for each $A \in |\mathcal{A}|$,
- A morphism $\eta_A : A \rightarrow TA$ for each $A \in |\mathcal{A}|$,
- A collection of maps $(-)^* : C(\Gamma \otimes (JA \otimes \Delta), TB) \rightarrow C(\Gamma \otimes (TA \otimes \Delta), TB)$ natural in $\Gamma, \Delta \in C$

all such that for $f : \Gamma \otimes (JA \otimes \Delta) \rightarrow TB$ and $g : \Gamma' \otimes (JB \otimes \Delta') \rightarrow TC$, the following equations hold:

$$\begin{aligned} (\eta_A \circ \rho_{JA} \circ \lambda_{JA \otimes I})^* &= \rho_{TA} \circ \lambda_{TA \otimes I} \\ f^* \circ (\Gamma \otimes (\eta_A \otimes \Delta)) &= f \\ g^* \circ (\Gamma' \otimes (f^* \otimes \Delta')) \circ \tilde{\alpha}_{\Gamma', \Gamma, TA, \Delta, \Delta'} &= (g^* \circ (\Gamma' \otimes (f \otimes \Delta'))) \circ \tilde{\alpha}_{\Gamma', \Gamma, JA, \Delta, \Delta'}^* \end{aligned}$$

where $\tilde{\alpha}_{\Gamma', \Gamma, X, \Delta, \Delta'} = (\Gamma' \otimes (\alpha_{\Gamma, X \otimes \Delta, \Delta'}^{-1} \circ (\Gamma \otimes \alpha_{X, \Delta, \Delta'}^{-1}))) \circ \alpha_{\Gamma', \Gamma, X \otimes (\Delta \otimes \Delta')}$.

When C is symmetric with symmetry σ , a bistrong J -relative monad is additionally said to be *symmetric* when for $f : (\Gamma \otimes \Delta) \otimes (JA \otimes \Gamma') \rightarrow TB$,

$$f^* \circ \tilde{\sigma}_{\Gamma, TA, \Delta, \Gamma'} = (f \circ \tilde{\sigma}_{\Gamma, JA, \Delta, \Gamma'})^*$$

where $\tilde{\sigma}_{\Gamma, X, \Delta, \Gamma'} : \Gamma \otimes (X \otimes (\Delta \otimes \Gamma')) \rightarrow (\Gamma \otimes \Delta) \otimes (X \otimes \Gamma')$ is given by

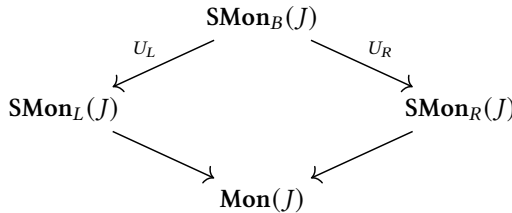
$$\tilde{\sigma}_{\Gamma, X, \Delta, \Gamma'} = \alpha_{\Gamma, \Delta, X \otimes \Gamma'}^{-1} \circ (\Gamma \otimes (\alpha_{\Delta, X, \Gamma'} \circ (\sigma_{X, \Delta} \otimes \Gamma') \circ \alpha_{X, \Delta, \Gamma'}^{-1})).$$

A *morphism of (symmetrically) bistrong J -relative monads* $\gamma : (T, \eta, (-)^*) \rightarrow (S, \nu, (-)^\dagger)$ consists of a family of maps $\gamma_A : TA \rightarrow SA$ satisfying, for any $f : \Gamma \otimes (JA \otimes \Delta) \rightarrow TB$,

$$\nu_A = \gamma_A \circ \eta_A \quad \gamma_B \circ f^* = (\gamma_B \circ f)^\dagger \circ (\Gamma \otimes (\gamma_A \otimes \Delta))$$

We now write $\mathbf{SMon}_B(J)$, $\mathbf{SMon}_L(J)$, and $\mathbf{SMon}_R(J)$ for the categories of bistrong, left strong, and right strong J -relative monads and morphisms, respectively.

There are forgetful functors:



which commute with the *underlying J -relative monad* functors. When C is symmetric monoidal, and we restrict $\mathbf{SMon}_B(J)$ to the full subcategory of symmetrically bistrong J -relative monads, U_L and U_R become isomorphisms.

Remark. For a jointly left and right strong J -relative monad $(T, \eta, (-)^{*L}, (-)^{*R})$ (such that the underlying J -relative monads coincide), we can formulate a compatibility condition for any $f : \Gamma \otimes JA \rightarrow JB$, $g : JA \otimes \Delta \rightarrow JC$, $h : JB \otimes \Delta \rightarrow TD$, and $k : \Gamma \otimes JC \rightarrow TD$:

$$\begin{array}{ccc}
 (\Gamma \otimes JA) \otimes \Delta \xrightarrow{\alpha_{\Gamma, JA, \Delta}} \Gamma \otimes (JA \otimes \Delta) & & (\Gamma \otimes TA) \otimes \Delta \xrightarrow{\alpha_{\Gamma, TA, \Delta}} \Gamma \otimes (TA \otimes \Delta) \\
 \downarrow f \otimes \Delta & \downarrow \Gamma \otimes g & \downarrow \Gamma \otimes (\eta_C \circ g)^{*R} \\
 \Gamma \otimes JC & \downarrow k & \Gamma \otimes TC \\
 \downarrow & & \downarrow k^{*L} \\
 JB \otimes \Delta \xrightarrow{h} TD & \Rightarrow & TB \otimes \Delta \xrightarrow{h^{*R}} TD
 \end{array}$$

Any bistrong J -relative monad satisfies this condition, and when $J = \text{id}_C$, this recovers the definition of a bistrong monad, but this is not generally sufficient to define the structure of a bistrong J -relative monad. For linear type-theoretic applications, Definition A.2 is more suitable, as it allows computations to be sequenced anywhere in a context.

The notion of J -strength can also be extended to J -bistrength in a canonical way. We note this definition because Proposition 4.14 holds at the generality of (symmetric) bistrong monads and J -relative monads with a (symmetric) J -bistrength.

Definition A.3. Given a J -relative monad $(T, \eta, (-)^{*})$, a left J -strength θ^L , and right J -strength θ^R (i.e. a left J -strength on C^{rev}), we say (θ^L, θ^R) is a J -bistrength for T if the following diagram commutes.

$$\begin{array}{ccccc}
 (JA \otimes TB) \otimes JC & \xrightarrow{\theta_{A,B}^L \otimes JC} & T(A \otimes B) \otimes JC & \xrightarrow{\theta_{A \otimes B, C}^R} & T((A \otimes B) \otimes C) \\
 \downarrow \alpha_{JA, TB, JC} & & & & \downarrow T\alpha_{A,B,C} \\
 JA \otimes (TB \otimes JC) & \xrightarrow{JA \otimes \theta_{B,C}^R} & JA \otimes T(B \otimes C) & \xrightarrow{\theta_{A, B \otimes C}^L} & T(A \otimes (B \otimes C))
 \end{array}$$

If $\mathcal{A}$ and $\mathcal{C}$ are symmetric monoidal categories and J is lax symmetric monoidal, we say T has *symmetric J -bistrength* if the following diagram commutes.

$$\begin{array}{ccc}
 JA \otimes TB & \xrightarrow{\theta_{A,B}^L} & T(A \otimes B) \\
 \downarrow \sigma_{JA, TB} & & \downarrow T\sigma_{A,B} \\
 TB \otimes JA & \xrightarrow{\theta_{A,B}^R} & T(B \otimes A)
 \end{array}$$

B Full term calculi and their equational theories

We require the following reflexivity, symmetry, transitivity, and substitution laws to hold for all judgements for the remainder of this appendix:

$$\begin{array}{c}
 \frac{\Gamma \vdash u : X}{\Gamma \vdash u = u : X} \quad \frac{\Gamma \vdash u = v : X}{\Gamma \vdash v = u : X} \quad \frac{\Gamma \vdash u = v : X \quad \Gamma \vdash v = w : X}{\Gamma \vdash u = w : X} \\
 \\
 \frac{\Gamma \vdash u : X \quad \Gamma, x : X \vdash s = t : Y}{\Gamma \vdash s[u/x] = t[u/x] : Y}
 \end{array}$$

We also require that all term constructors satisfy *congruence* equations, e.g. for product types:

$$\frac{\Gamma \vdash u = v : X \times Y}{\Gamma \vdash \pi_1 u = \pi_1 v : X} \quad \frac{\Gamma \vdash u = v : X \times Y}{\Gamma \vdash \pi_2 u = \pi_2 v : Y} \quad \frac{\Gamma \vdash u = v : X \quad \Gamma \vdash t = s : Y}{\Gamma \vdash (u, t) = (v, s) : X \times Y}$$

B.1 Relative monadic metalanguage (RMM)

The types and contexts of RMM are:

$$\begin{array}{ll} \text{Types:} & X, Y ::= 1 \mid X \times Y \mid JA \mid TA \\ \text{Contexts:} & \Gamma ::= \emptyset \mid \Gamma, x : X \end{array} \quad (A \in |\mathcal{A}|)$$

The terms are:

$$\begin{array}{c} \frac{}{\Gamma, x : X \vdash x : X} \quad \frac{}{\Gamma \vdash () : 1} \quad \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash \pi_1 u : X} \quad \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash \pi_2 u : Y} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash (u, t) : X \times Y} \\ \\ \frac{\Gamma \vdash u : JA}{\Gamma \vdash f u : JB} (f \in \mathcal{A}(A, B)) \quad \frac{\Gamma \vdash u : JA}{\Gamma \vdash \text{return } u : TA} \quad \frac{\Gamma \vdash u : TA \quad \Gamma, y : JA \vdash t : TB}{\Gamma \vdash \text{do } y \leftarrow u \text{ in } t : TB} \end{array}$$

The additional equations beyond the symmetry, reflexivity, transitivity, congruence and substitution laws are:

$$\begin{array}{c} \frac{\Gamma \vdash u : 1}{\Gamma \vdash u = () : 1} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash \pi_1(u, t) = u : X} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash \pi_2(u, t) = t : Y} \\ \\ \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash (\pi_1 u, \pi_2 u) = u : X \times Y} \quad \frac{\Gamma \vdash u : JA}{\Gamma \vdash \text{id}_A u = u : JA} \\ \\ \frac{\Gamma \vdash u : JA}{\Gamma \vdash (g \circ f) u = g(f u) : JC} (f \in \mathcal{A}(A, B), g \in \mathcal{A}(B, C)) \\ \\ \frac{\Gamma \vdash u : JA \quad \Gamma, y : JA \vdash t : TB}{\Gamma \vdash \text{do } y \leftarrow \text{return } u \text{ in } t = t[u/y] : TB} \quad \frac{\Gamma \vdash u : TA}{\Gamma \vdash \text{do } y \leftarrow u \text{ in return } y = u : TA} \\ \\ \frac{\Gamma \vdash u : TA \quad \Gamma, y : JA \vdash t : TB \quad \Gamma, z : JB \vdash v : TC}{\Gamma \vdash \text{do } z \leftarrow (\text{do } y \leftarrow u \text{ in } t) \text{ in } v = \text{do } y \leftarrow u \text{ in } (\text{do } z \leftarrow t \text{ in } v) : TC} \end{array}$$

B.2 Graded monadic metalanguage (GMM)

The types and contexts of GMM are:

$$\begin{array}{ll} \text{Types:} & X, Y ::= 1 \mid X \times Y \mid T_m X \\ \text{Contexts:} & \Gamma ::= \emptyset \mid \Gamma, x : X \end{array} \quad (m \in |\mathcal{M}|)$$

The terms are:

$$\begin{array}{c}
\frac{}{\Gamma, x : X \vdash x : X} \quad \frac{}{\Gamma \vdash () : 1} \quad \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash \pi_1 u : X} \quad \frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash \pi_2 u : Y} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash (u, t) : X \times Y} \\
\\
\frac{\Gamma \vdash u : X}{\Gamma \vdash \text{return } u : T_e X} \quad \frac{\Gamma \vdash u : T_m X \quad \Gamma, x : X \vdash t : T_n Y}{\Gamma \vdash \text{do } x \leftarrow u \text{ in } t : T_{m \oplus n} Y} \quad \frac{\Gamma \vdash u : T_n X}{\Gamma \vdash T_\xi X u : T_m X} (\xi \in \mathcal{M}(m, n))
\end{array}$$

The additional equations beyond the symmetry, reflexivity, transitivity, congruence and substitution laws are:

$$\begin{array}{c}
\frac{\Gamma \vdash u : 1}{\Gamma \vdash u = () : 1} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash \pi_1(u, t) = u : X} \quad \frac{\Gamma \vdash u : X \quad \Gamma \vdash t : Y}{\Gamma \vdash \pi_2(u, t) = t : Y} \\
\\
\frac{\Gamma \vdash u : X \times Y}{\Gamma \vdash (\pi_1 u, \pi_2 u) = u : X \times Y} \quad \frac{\Gamma \vdash u : T_m X}{\Gamma \vdash T_{\text{id}_m} X u = u : T_m X} \\
\\
\frac{\Gamma \vdash u : T_l X}{\Gamma \vdash T_{\varphi \circ \xi} X u = T_\xi X (T_\varphi X u) : T_m X} (\xi \in \mathcal{M}(m, n), \varphi \in \mathcal{M}(n, l)) \\
\\
\frac{\Gamma \vdash u : T_l X \quad \Gamma, x : X \vdash t : T_n Y}{\Gamma \vdash \text{do } x \leftarrow u \text{ in } (T_\xi Y t) = T_{l \oplus \xi} Y (\text{do } x \leftarrow u \text{ in } t) : T_{l \oplus m} Y} (\xi \in \mathcal{M}(m, n)) \\
\\
\frac{\Gamma \vdash u : T_n X \quad \Gamma, x : X \vdash t : T_l Y}{\Gamma \vdash \text{do } x \leftarrow (T_\xi X u) \text{ in } t = T_{\xi \oplus l} Y (\text{do } x \leftarrow u \text{ in } t) : T_{m \oplus l} Y} (\xi \in \mathcal{M}(m, n)) \\
\\
\frac{\Gamma \vdash u : X \quad \Gamma, x : X \vdash t : T_m Y}{\Gamma \vdash \text{do } x \leftarrow \text{return } u \text{ in } t = T_{\lambda_m} Y t[u/x] : T_{e \oplus m} Y} \quad \frac{\Gamma \vdash u : T_m X}{\Gamma \vdash \text{do } x \leftarrow u \text{ in return } x = T_{\rho_m} X u : T_{m \oplus e} X} \\
\\
\frac{\Gamma \vdash u : T_l X \quad \Gamma, x : X \vdash t : T_m Y \quad \Gamma, y : Y \vdash v : T_n Z}{\Gamma \vdash \text{do } y \leftarrow (\text{do } x \leftarrow u \text{ in } t) \text{ in } v = T_{\alpha_{l, m, n}} Z (\text{do } x \leftarrow u \text{ in } (\text{do } y \leftarrow t \text{ in } v)) : T_{(l \oplus m) \oplus n} Z}
\end{array}$$

B.3 Linear-non-linear RMM (LNL-RMM)

The types and contexts of LNL-RMM are:

$$\begin{array}{ll}
\mathcal{A} \text{ types:} & A, B ::= 1 \mid A \times B \mid A \rightarrow B \mid RX \\
\mathcal{C} \text{ types:} & X, Y ::= I \mid \underline{m} \mid X \otimes Y \mid X \multimap Y \mid JA \mid TA \quad (m \in |\mathcal{M}|) \\
\mathcal{A} \text{ contexts:} & \Gamma, ::= \emptyset \mid \Gamma, a : A \\
\mathcal{C} \text{ contexts:} & \Delta, ::= \emptyset \mid \Delta, x : X
\end{array}$$

The non-linear terms for $\mathcal{A}$ types are:

$$\begin{array}{c}
\frac{}{\Gamma, a : A \vdash_{\mathcal{A}} a : A} \quad \frac{}{\Gamma \vdash_{\mathcal{A}} () : 1} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \times B}{\Gamma \vdash_{\mathcal{A}} \pi_1 u : A} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \times B}{\Gamma \vdash_{\mathcal{A}} \pi_2 u : B} \\
\\
\frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma \vdash_{\mathcal{A}} v : B}{\Gamma \vdash_{\mathcal{A}} (u, v) : A \times B} \quad \frac{\Gamma, a : A \vdash_{\mathcal{A}} u : B}{\Gamma \vdash_{\mathcal{A}} \lambda(a : A).u : A \rightarrow B} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \rightarrow B \quad \Gamma \vdash_{\mathcal{A}} v : A}{\Gamma \vdash_{\mathcal{A}} u v : B}
\end{array}$$

The linear terms for $\mathcal{C}$ types are:

$$\begin{array}{c}
\frac{}{\Gamma; x : X \vdash_C x : X} \quad \frac{}{\Gamma; \emptyset \vdash_C () : I} \quad \frac{\Gamma; \Delta_1 \vdash_C t : X \quad \Gamma; \Delta_2 \vdash_C s : Y}{\Gamma; \Delta_1, \Delta_2 \vdash_C (t, s) : X \otimes Y} \\
\\
\frac{\Gamma; \Delta_1 \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : X}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } () \text{ be } t \text{ in } s : X} \quad \frac{\Gamma; \Delta_1 \vdash_C t : X \otimes Y \quad \Gamma; \Delta_2, x : X, y : Y \vdash_C s : Z}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } (x, y) \text{ be } t \text{ in } s : Z} \\
\\
\frac{\Gamma; \Delta, x : X \vdash_C s : Y}{\Gamma; \Delta \vdash_C \lambda(x : X).s : X \multimap Y} \quad \frac{\Gamma; \Delta_1 \vdash_C s : X \multimap Y \quad \Gamma; \Delta_2 \vdash_C t : X}{\Gamma; \Delta_1, \Delta_2 \vdash_C s t : Y} \quad \frac{\Gamma; \Delta \vdash_C t : JA}{\Gamma; \Delta \vdash_C \text{return } t : TA} \\
\\
\frac{\Gamma; \Delta_1 \vdash_C t : TA \quad \Gamma; \Delta_2, y : JA \vdash_C s : TB}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{do } y \leftarrow t \text{ in } s : TB} \quad \frac{\Gamma; \Delta \vdash_C t : \underline{m} \quad (\xi \in \mathcal{M}(m, n))}{\Gamma; \Delta \vdash_C \underline{\xi} t : \underline{n}} \\
\\
\frac{\Gamma; \Delta \vdash_C t : \underline{e}}{\Gamma; \Delta \vdash_C \text{unmerge } t : I} \quad \frac{\Gamma; \Delta \vdash_C t : I}{\Gamma; \Delta \vdash_C \text{merge } t : \underline{e}} \quad \frac{\Gamma; \Delta \vdash_C t : \underline{m} \oplus \underline{n}}{\Gamma; \Delta \vdash_C \text{unmerge } t : \underline{m} \otimes \underline{n}} \\
\\
\frac{\Gamma; \Delta \vdash_C t : \underline{m} \otimes \underline{n}}{\Gamma; \Delta \vdash_C \text{merge } t : \underline{m} \oplus \underline{n}}
\end{array}$$

The terms for linear-non-linear interaction are:

$$\begin{array}{c}
\frac{\Gamma \vdash_{\mathcal{A}} u : A}{\Gamma; \emptyset \vdash_C J(u) : JA} \quad \frac{\Gamma; \Delta_1 \vdash_C t : JA \quad \Gamma, a : A; \Delta_2 \vdash_C s : X}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } J(a) \text{ be } t \text{ in } s : X} \quad \frac{\Gamma; \emptyset \vdash_C t : X}{\Gamma \vdash_{\mathcal{A}} R(t) : RX} \\
\\
\frac{\Gamma \vdash_{\mathcal{A}} u : RX}{\Gamma; \emptyset \vdash_C \text{derelict } u : X}
\end{array}$$

The equations for terms in $\mathcal{A}$ are:

$$\begin{array}{c}
\frac{\Gamma \vdash_{\mathcal{A}} u : 1}{\Gamma \vdash_{\mathcal{A}} u = () : 1} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma \vdash_{\mathcal{A}} v : B}{\Gamma \vdash_{\mathcal{A}} \pi_1(u, v) = u : A} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma \vdash_{\mathcal{A}} v : B}{\Gamma \vdash_{\mathcal{A}} \pi_2(u, v) = v : B} \\
\\
\frac{\Gamma \vdash_{\mathcal{A}} u : A \times B}{\Gamma \vdash_{\mathcal{A}} (\pi_1 u, \pi_2 u) = u : A \times B} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma, a : A \vdash_{\mathcal{A}} v : B}{\Gamma \vdash_{\mathcal{A}} (\lambda(a : A).v) u = v[u/a] : B} \\
\\
\frac{\Gamma \vdash_{\mathcal{A}} u : A \rightarrow B}{\Gamma \vdash_{\mathcal{A}} \lambda(a : A).u a = u : A \rightarrow B}
\end{array}$$

The equations for the linear lambda calculus terms in C are:

$$\begin{array}{c}
\frac{\Gamma; \Delta \vdash_C t : X}{\Gamma; \Delta \vdash_C \text{let } () \text{ be } () \text{ in } t = t : X} \quad \frac{\Gamma; \Delta \vdash_C t : I}{\Gamma; \Delta \vdash_C \text{let } () \text{ be } t \text{ in } () = t : I} \\
\\
\frac{\Gamma; \Delta_1 \vdash_C t : X \quad \Gamma; \Delta_2 \vdash_C s : Y \quad \Gamma; \Delta_3, x : X, y : Y \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } (x, y) \text{ be } (t, s) \text{ in } r = r[t/x, s/y] : Z} \\
\\
\frac{\Gamma; \Delta \vdash_C t : X \otimes Y}{\Gamma; \Delta \vdash_C \text{let } (x, y) \text{ be } (x, y) \text{ in } t = t : X \otimes Y} \quad \frac{\Gamma; \Delta_1, x : X \vdash_C s : Y \quad \Gamma; \Delta_2 \vdash_C t : X}{\Gamma; \Delta_1, \Delta_2 \vdash_C (\lambda(x : X).s) t = s[t/x] : Y}
\end{array}$$

$$\frac{\Gamma; \Delta \vdash_C t : X \multimap Y}{\Gamma; \Delta \vdash_C \lambda(x : X).tx = t : X \multimap Y} \quad \frac{\Gamma; \Delta \vdash_C t : X \multimap Y}{\Gamma; \Delta \vdash_C \lambda(x : X).tx = t : X \multimap Y}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : W \multimap Z \quad \Gamma; \Delta_2 \vdash_C s : I \quad \Gamma; \Delta_3 \vdash_C r : W}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C (\text{let } () \text{ be s in } t) r = \text{let } () \text{ be s in } (tr) : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : W \multimap Z \quad \Gamma; \Delta_2 \vdash_C s : I \quad \Gamma; \Delta_3 \vdash_C r : W}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C t (\text{let } () \text{ be s in } r) = \text{let } () \text{ be s in } (tr) : Z}$$

$$\frac{\Gamma; \Delta_1, x : X, y : Y \vdash_C t : W \multimap Z \quad \Gamma; \Delta_2 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_3 \vdash_C r : W}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C (\text{let } (x, y) \text{ be s in } t) r = \text{let } (x, y) \text{ be s in } (tr) : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : W \multimap Z \quad \Gamma; \Delta_2 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_3, x : X, y : Y \vdash_C r : W}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C t (\text{let } (x, y) \text{ be s in } r) = \text{let } (x, y) \text{ be s in } (tr) : Z}$$

$$\frac{\Gamma; \Delta_1, x : X \vdash_C t : Y \quad \Gamma; \Delta_2 \vdash_C s : I}{\Gamma; \Delta_1, \Delta_2 \vdash_C \lambda(x : X).(\text{let } () \text{ be s in } t) = \text{let } () \text{ be s in } (\lambda(x : X).t) : X \multimap Y}$$

$$\frac{\Gamma; \Delta_1, w : W, z : Z, x : X \vdash_C t : Y \quad \Gamma; \Delta_2 \vdash_C s : W \otimes Z}{\Gamma; \Delta_1, \Delta_2 \vdash_C \lambda(x : X).(\text{let } (w, z) \text{ be s in } t) = \text{let } (w, z) \text{ be s in } (\lambda(x : X).t) : X \multimap Y}$$

$$\frac{\Gamma; \Delta_1, x : X, y : Y, w : W \vdash_C t : Z \quad \Gamma; \Delta_2 \vdash_C s : X \otimes Y}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C t (\text{let } (x, y) \text{ be s in } r) = \text{let } (x, y) \text{ be s in } (tr) : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : I \quad \Gamma; \Delta_3 \vdash_C r : X}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } () \text{ be } (\text{let } () \text{ be } t \text{ in } s) \text{ in } r = \text{let } () \text{ be } t \text{ in } (\text{let } () \text{ be s in } r) : X}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_3, x : X, y : Y \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } (x, y) \text{ be } (\text{let } () \text{ be } t \text{ in } s) \text{ in } r = \text{let } () \text{ be } t \text{ in } (\text{let } (x, y) \text{ be s in } r) : Z}$$

$$\frac{\Gamma; \Delta_1, x : X, y : Y \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_3 \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } (x, y) \text{ be s in } (\text{let } () \text{ be } t \text{ in } r) = \text{let } () \text{ be } (\text{let } (x, y) \text{ be s in } t) \text{ in } r : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : Z \otimes W \quad \Gamma; \Delta_2, z : Z, w : W \vdash_C s : X \otimes Y \quad \Gamma; \Delta_3, x : X, y : Y \vdash_C r : R}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } (x, y) \text{ be } (\text{let } (z, w) \text{ be } t \text{ in } s) \text{ in } r = \text{let } (z, w) \text{ be } t \text{ in } (\text{let } (x, y) \text{ be s in } r) : R}$$

The additional equations for RMM terms in C are:

$$\frac{\Gamma; \Delta_1 \vdash_C u : JA \quad \Gamma; \Delta_2, y : JA \vdash_C t : TB}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{do } y \leftarrow \text{return } u \text{ in } t = t[u/y] : TB} \quad \frac{\Gamma; \Delta \vdash_C u : TA}{\Gamma; \Delta \vdash_C \text{do } y \leftarrow u \text{ in return } y = u : TA}$$

$$\frac{\Gamma; \Delta_1 \vdash_C u : TA \quad \Gamma; \Delta_2, y : JA \vdash_C t : TB \quad \Gamma; \Delta_3, z : JB \vdash_C v : TC}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } z \leftarrow (\text{do } y \leftarrow u \text{ in } t) \text{ in } v = \text{do } y \leftarrow u \text{ in } (\text{do } z \leftarrow t \text{ in } v) : TC}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : I \quad \Gamma; \Delta_2 \vdash_C u : JA}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } () \text{ be } s \text{ in return } u = \text{return } (\text{let } () \text{ be } s \text{ in } u) : TA}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_2, x : X, y : Y \vdash_C u : JA}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } (x, y) \text{ be } s \text{ in return } u = \text{return } (\text{let } () \text{ be } (x, y) \text{ in } u) : TA}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : JA \quad \Gamma, a : A; \Delta_2 \vdash_C u : JB}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } J(a) \text{ be } s \text{ in return } u = \text{return } (\text{let } J(a) \text{ be } s \text{ in } u) : TB}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : I \quad \Gamma; \Delta_2 \vdash_C u : TA \quad \Gamma; \Delta_3, y : JA \vdash_C t : TB}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } y \leftarrow (\text{let } () \text{ be } s \text{ in } u) \text{ in } t = \text{let } () \text{ be } s \text{ in } (\text{do } y \leftarrow u \text{ in } t) : TB}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : I \quad \Gamma; \Delta_2 \vdash_C u : TA \quad \Gamma; \Delta_3, y : JA \vdash_C t : TB}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } y \leftarrow u \text{ in } (\text{let } () \text{ be } s \text{ in } t) = \text{let } () \text{ be } s \text{ in } (\text{do } y \leftarrow u \text{ in } t) : TB}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_2, x : X, y : Y \vdash_C u : TA \quad \Gamma; \Delta_3, z : JA \vdash_C t : TB}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } z \leftarrow (\text{let } (x, y) \text{ be } s \text{ in } u) \text{ in } t = \text{let } (x, y) \text{ be } s \text{ in } (\text{do } z \leftarrow u \text{ in } t) : TB}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : X \otimes Y \quad \Gamma; \Delta_2 \vdash_C u : TA \quad \Gamma; \Delta_3, x : X, y : Y, z : JA \vdash_C t : TB}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } z \leftarrow u \text{ in } (\text{let } (x, y) \text{ be } s \text{ in } t) = \text{let } (x, y) \text{ be } s \text{ in } (\text{do } z \leftarrow u \text{ in } t) : TB}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : JA \quad \Gamma, a : A; \Delta_2 \vdash_C u : TB \quad \Gamma; \Delta_3, y : JB \vdash_C t : TC}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } y \leftarrow (\text{let } J(a) \text{ be } s \text{ in } u) \text{ in } t = \text{let } J(a) \text{ be } s \text{ in } (\text{do } y \leftarrow u \text{ in } t) : TC}$$

$$\frac{\Gamma; \Delta_1 \vdash_C s : JA \quad \Gamma; \Delta_2 \vdash_C u : TB \quad \Gamma, a : A; \Delta_3, y : JB \vdash_C t : TC}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{do } y \leftarrow u \text{ in } (\text{let } J(a) \text{ be } s \text{ in } u) = \text{let } J(a) \text{ be } s \text{ in } (\text{do } y \leftarrow u \text{ in } t) : TC}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m}}{\Gamma; \Delta \vdash_C \underline{1_m} t = t : \underline{m}} \quad \frac{\Gamma; \Delta \vdash_C t : \underline{m}}{\Gamma; \Delta \vdash_C \underline{\varphi \circ \xi} t = \underline{\varphi} (\underline{\xi} t) : \underline{l}} (\xi \in \mathcal{M}(m, n), \varphi \in \mathcal{M}(n, l))$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{e}}{\Gamma; \Delta \vdash_C \text{merge unmerge } t = t : \underline{e}}$$

$$\frac{\Gamma; \Delta \vdash_C t : I}{\Gamma; \Delta \vdash_C \text{unmerge merge } t = t : I}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m \oplus n}}{\Gamma; \Delta \vdash_C \text{merge unmerge } t = t : \underline{m \oplus n}}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m \otimes n}}{\Gamma; \Delta \vdash_C \text{unmerge merge } t = t : \underline{m \otimes n}}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m \oplus n}}{\Gamma; \Delta \vdash_C \text{unmerge } (\underline{\xi \oplus \varphi} t) = \text{let } (s, r) \text{ be unmerge } t \text{ in } (\underline{\xi} s, \underline{\varphi} r) : \underline{m'} \otimes \underline{n'}} (\xi \in \mathcal{M}(m, m'), \varphi \in \mathcal{M}(n, n'))$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{e} \oplus \underline{m}}{\Gamma; \Delta \vdash_C \text{let } (s, r) \text{ be unmerge } t \text{ in } (\text{let } () \text{ be unmerge } s \text{ in } r) = \underline{\lambda_m} t : \underline{m}}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m} \oplus \underline{e}}{\Gamma; \Delta \vdash_C \text{let } (s, r) \text{ be unmerge } t \text{ in } (\text{let } () \text{ be unmerge } r \text{ in } s) = \underline{\rho_m} t : \underline{m}}$$

$$\frac{\Gamma; \Delta \vdash_C t : \underline{m} \oplus \underline{n}}{\Gamma; \Delta \vdash_C \text{let } (s, r) \text{ be unmerge } t \text{ in } (r, s) = \text{unmerge } (\underline{\sigma_{m,n}} t) : \underline{n} \otimes \underline{m}}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : (\underline{m} \oplus \underline{n}) \oplus \underline{l} \quad \Gamma; \Delta_2, x : \underline{m}, y : \underline{n}, z : \underline{l} \vdash_C u : \underline{m} \otimes (\underline{n} \otimes \underline{l})}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } (x, r) \text{ be unmerge } (\underline{\alpha_{m,n,l}} t) \text{ in } (\text{let } (y, z) \text{ be unmerge } r \text{ in } u) = \text{let } (s, z) \text{ be unmerge } t \text{ in } (\text{let } (x, y) \text{ be unmerge } s \text{ in } u) : Z}$$

The equations for linear-non-linear interactions are:

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma, a : A; \Delta \vdash_C t : X}{\Gamma; \Delta \vdash_C \text{let } J(a) \text{ be } J(u) \text{ in } t = t[u/a] : X} \quad \frac{\Gamma; \Delta \vdash_C t : JA}{\Gamma; \Delta \vdash_C \text{let } J(a) \text{ be } t \text{ in } J(a) = t : JA}$$

$$\frac{\Gamma; \emptyset \vdash_C t : X}{\Gamma; \emptyset \vdash_C \text{derelict } R(t) = t : X} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : RX}{\Gamma \vdash_{\mathcal{A}} R(\text{derelict } u) = u : RX}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma, a : A; \Delta_3 \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(a) \text{ be } (\text{let } () \text{ be } t \text{ in } s) \text{ in } r = \text{let } () \text{ be } t \text{ in } (\text{let } J(a) \text{ be } s \text{ in } r) : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma, a : A; \Delta_3 \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(a) \text{ be } s \text{ in } (\text{let } () \text{ be } t \text{ in } r) = \text{let } () \text{ be } t \text{ in } (\text{let } J(a) \text{ be } s \text{ in } r) : Z}$$

$$\frac{\Gamma, a : A; \Delta_1 \vdash_C t : I \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma; \Delta_3 \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(a) \text{ be } s \text{ in } (\text{let } () \text{ be } t \text{ in } r) = \text{let } () \text{ be } (\text{let } J(a) \text{ be } s \text{ in } t) \text{ in } r : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : X \otimes Y \quad \Gamma; \Delta_2, x : X, y : Y \vdash_C s : JA \quad \Gamma, a : A; \Delta_3 \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(a) \text{ be } (\text{let } (x, y) \text{ be } t \text{ in } s) \text{ in } r = \text{let } (x, y) \text{ be } t \text{ in } (\text{let } J(a) \text{ be } s \text{ in } r) : Z}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : X \otimes Y \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma, a : A; \Delta_3, x : X, y : Y \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(a) \text{ be } s \text{ in } (\text{let } (x, y) \text{ be } t \text{ in } r) = \text{let } (x, y) \text{ be } t \text{ in } (\text{let } J(a) \text{ be } s \text{ in } r) : Z}$$

$$\frac{\Gamma, a : A; \Delta_1 \vdash_C t : X \otimes Y \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma; \Delta_3, x : X, y : Y \vdash_C r : Z}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(a) \text{ be } s \text{ in } (\text{let } (x, y) \text{ be } t \text{ in } r) = \text{let } (x, y) \text{ be } (\text{let } J(a) \text{ be } s \text{ in } t) \text{ in } r : Z}$$

$$\frac{\Gamma, a : A; \Delta_1 \vdash_C t : X \multimap Y \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma; \Delta_3 \vdash_C r : X}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C (\text{let } J(a) \text{ be } s \text{ in } t) r = \text{let } J(a) \text{ be } s \text{ in } (t r) : Y}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : X \multimap Y \quad \Gamma; \Delta_2 \vdash_C s : JA \quad \Gamma; \Delta_3, a : A \vdash_C r : X}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C t (\text{let } J(a) \text{ be } s \text{ in } r) = \text{let } J(a) \text{ be } s \text{ in } (t r) : Y}$$

$$\frac{\Gamma, a : A; \Delta_1, x : X \vdash_C t : Y \quad \Gamma; \Delta_2 \vdash_C s : JA}{\Gamma; \Delta_1, \Delta_2 \vdash_C \text{let } J(a) \text{ be } s \text{ in } \lambda(x : X).t = \lambda(x : X).(\text{let } J(a) \text{ be } s \text{ in } t) : X \multimap Y}$$

$$\frac{\Gamma; \Delta_1 \vdash_C t : JA \quad \Gamma, a : A; \Delta_2 \vdash_C s : JB \quad \Gamma, b : B; \Delta_3 \vdash_C r : X}{\Gamma; \Delta_1, \Delta_2, \Delta_3 \vdash_C \text{let } J(b) \text{ be } (\text{let } J(a) \text{ be } t \text{ in } s) \text{ in } r = \text{let } J(a) \text{ be } t \text{ in } (\text{let } J(b) \text{ be } s \text{ in } r) : X}$$

B.4 Arrow relative monadic metalanguage (ARMM)

The types and contexts of ARMM are:

$$\begin{aligned} \mathcal{A} \text{ types:} \quad & A, B ::= 1 \mid A \times B \mid A \Rightarrow B \\ C \text{ types:} \quad & X, Y ::= 1 \mid X \times Y \mid JA \mid KA \mid TA \\ \mathcal{A} \text{ contexts:} \quad & \Gamma, \Delta ::= \emptyset \mid \Gamma, a : A \\ C \text{ contexts:} \quad & \Phi ::= \emptyset \mid \Phi, x : X \end{aligned}$$

The terms for $\mathcal{A}$ types are:

$$\begin{aligned} \frac{}{\Gamma, a : A \vdash_{\mathcal{A}} a : A} \quad & \frac{}{\Gamma \vdash_{\mathcal{A}} () : 1} \quad & \frac{\Gamma \vdash_{\mathcal{A}} u : A \times B}{\Gamma \vdash \pi_1 u : A} \quad & \frac{\Gamma \vdash_{\mathcal{A}} u : A \times B}{\Gamma \vdash \pi_2 u : B} \\ & \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma \vdash v : B}{\Gamma \vdash (u, v) : A \times B} \end{aligned}$$

The terms for C types are:

$$\begin{aligned} \frac{}{\Gamma; \Delta; \Phi, x : X \vdash_C x : X} \quad & \frac{}{\Gamma; \Delta; \Phi \vdash_C () : 1} \quad & \frac{\Gamma; \Delta; \Phi \vdash_C t : X \times Y}{\Gamma; \Delta; \Phi \vdash_C \pi_1 t : X} \quad & \frac{\Gamma; \Delta; \Phi \vdash_C t : X \times Y}{\Gamma; \Delta; \Phi \vdash_C \pi_2 t : Y} \\ & \frac{\Gamma; \Delta; \Phi \vdash_C t : X \quad \Gamma; \Delta; \Phi \vdash_C s : Y}{\Gamma; \Delta; \Phi \vdash_C (t, s) : X \times Y} \quad & \frac{\Gamma; \Delta; \Phi \vdash_C t : JA \quad \Gamma; \Delta, a : A; \Phi \vdash_C s : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } t \text{ in } s : X} \\ & \frac{\Gamma; \Delta; \Phi \vdash_C t : KA \quad \Gamma, a : A; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } t \text{ in } s : X} \quad & \frac{\Gamma; \Delta; \Phi \vdash_C t : JA}{\Gamma; \Delta; \Phi \vdash_C \text{return } t : TA} \\ & \frac{\Gamma; \Delta; \Phi \vdash_C t : TA \quad \Gamma; \Delta; x : JA \vdash_C s : TB}{\Gamma; \Delta; \Phi \vdash_C \text{do } x \leftarrow t \text{ in } s : TB} \end{aligned}$$

The terms for the interaction between $\mathcal{A}$ and C are:

$$\begin{aligned} \frac{\Gamma \vdash_{\mathcal{A}} u : A \Rightarrow X \quad \Gamma, \Delta \vdash_{\mathcal{A}} v : A}{\Gamma; \Delta; \Phi \vdash_C u \bullet v : X} \quad & \frac{\Gamma; x : A; \emptyset \vdash_C t : X}{\Gamma \vdash_{\mathcal{A}} \lambda(x : A).t : A \Rightarrow X} \\ & \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma, \Delta \vdash_{\mathcal{A}} u : A}{\Gamma; \Delta; \Phi \vdash_C K(u) : KA} \quad & \frac{\Gamma, \Delta \vdash_{\mathcal{A}} u : A}{\Gamma; \Delta; \Phi \vdash_C J(u) : JA} \quad & \frac{\Gamma \vdash_{\mathcal{A}} u : A \Rightarrow JB \quad \Gamma \vdash_{\mathcal{A}} v : A}{\Gamma \vdash_{\mathcal{A}} u v : B} \end{aligned}$$

The equations for terms in $\mathcal{A}$ are:

$$\begin{array}{c}
\frac{\Gamma \vdash_{\mathcal{A}} u : 1}{\Gamma \vdash_{\mathcal{A}} u = () : 1} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma \vdash_{\mathcal{A}} v : B}{\Gamma \vdash_{\mathcal{A}} \pi_1(u, v) = u : A} \quad \frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma \vdash_{\mathcal{A}} v : B}{\Gamma \vdash_{\mathcal{A}} \pi_2(u, v) = t : B} \\
\\
\frac{\Gamma \vdash_{\mathcal{A}} u : A \times B}{\Gamma \vdash_{\mathcal{A}} (\pi_1 u, \pi_2 u) = u : A \times B}
\end{array}$$

The equations for terms in C are:

$$\begin{array}{c}
\frac{\Gamma; \Delta; \Phi \vdash_C t : 1}{\Gamma; \Delta; \Phi \vdash_C t = () : 1} \quad \frac{\Gamma; \Delta; \Phi \vdash_C t : X \quad \Gamma; \Delta; \Phi \vdash_C s : Y}{\Gamma; \Delta; \Phi \vdash_C \pi_1(t, s) = t : X} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C t : X \quad \Gamma; \Delta; \Phi \vdash_C s : Y}{\Gamma; \Delta; \Phi \vdash_C \pi_2(t, s) = s : Y} \quad \frac{\Gamma; \Delta; \Phi \vdash_C t : X \times Y}{\Gamma; \Delta; \Phi \vdash_C (\pi_1 t, \pi_2 t) = t : X \times Y} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C t : JA \quad \Gamma; \Delta; x : JA \vdash_C s : TB}{\Gamma; \Delta; \Phi \vdash_C \text{do } x \leftarrow \text{return } t \text{ in } s = s[t/x] : TB} \quad \frac{\Gamma; \Delta; \Phi \vdash_C t : TA}{\Gamma; \Delta; \Phi \vdash_C \text{do } x \leftarrow t \text{ in return } x = t : TB} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : TA \quad \Gamma; \Delta; x : JA \vdash_C s : TB \quad \Gamma; \Delta; y : JB \vdash_C t : TC}{\Gamma; \Delta; \Phi \vdash_C \text{do } x \leftarrow r \text{ in } (\text{do } y \leftarrow s \text{ in } t) = \text{do } y \leftarrow (\text{do } x \leftarrow r \text{ in } s) \text{ in } t : TC} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : JA \quad \Gamma; \Delta, a : A; \Phi \vdash_C s : JB \quad \Gamma; \Delta, b : B; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } r \text{ in } (\text{let } J(b) \text{ be } s \text{ in } t) = \text{let } J(b) \text{ be } (\text{let } J(a) \text{ be } r \text{ in } s) \text{ in } t : X} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : KA \quad \Gamma, a : A; \Delta; \Phi \vdash_C s : KB \quad \Gamma, b : B; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } r \text{ in } (\text{let } K(b) \text{ be } s \text{ in } t) = \text{let } K(b) \text{ be } (\text{let } K(a) \text{ be } r \text{ in } s) \text{ in } t : X} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : JA \quad \Gamma; \Delta, a : A; \Phi \vdash_C s : KB \quad \Gamma, b : B; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } r \text{ in } (\text{let } K(b) \text{ be } s \text{ in } t) = \text{let } K(b) \text{ be } (\text{let } J(a) \text{ be } r \text{ in } s) \text{ in } t : X} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : KA \quad \Gamma, a : A; \Delta; \Phi \vdash_C s : JB \quad \Gamma; \Delta, b : B; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } r \text{ in } (\text{let } J(b) \text{ be } s \text{ in } t) = \text{let } J(b) \text{ be } (\text{let } K(a) \text{ be } r \text{ in } s) \text{ in } t : X} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C u : J1}{\Gamma; \Delta; \Phi \vdash_C u = J() : J1} \quad \frac{\Gamma; \Delta; \Phi \vdash_C u : K1}{\Gamma; \Delta; \Phi \vdash_C u = K() : K1} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : JA \quad \Gamma; \Delta, a : A; \Phi \vdash_C s : X \quad \Gamma; \Delta, a : A; \Phi \vdash_C t : Y}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } r \text{ in } (s, t) = (\text{let } J(a) \text{ be } r \text{ in } s, \text{let } J(a) \text{ be } r \text{ in } t) : X \times Y} \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C s : JA \quad \Gamma; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } s \text{ in } t = t : X} (a : A \notin \text{fv}(t)) \\
\\
\frac{\Gamma; \Delta; \Phi \vdash_C r : KA \quad \Gamma, a : A; \Delta; \Phi \vdash_C s : X \quad \Gamma, a : A; \Delta; \Phi \vdash_C t : Y}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } r \text{ in } (s, t) = (\text{let } K(a) \text{ be } r \text{ in } s, \text{let } K(a) \text{ be } r \text{ in } t) : X \times Y}
\end{array}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C s : KA \quad \Gamma; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } s \text{ in } t = t : X} (a : A \notin \text{fv}(t))$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C r : JA \quad \Gamma; \Delta; \Phi \vdash_C s : JB \quad \Gamma; \Delta, a : A, b : B; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } r \text{ in } (\text{let } J(b) \text{ be } s \text{ in } t) = \text{let } J(b) \text{ be } s \text{ in } (\text{let } J(a) \text{ be } r \text{ in } t) : X}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C r : KA \quad \Gamma; \Delta; \Phi \vdash_C s : KB \quad \Gamma, a : A, b : B; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } r \text{ in } (\text{let } K(b) \text{ be } s \text{ in } t) = \text{let } K(b) \text{ be } s \text{ in } (\text{let } K(a) \text{ be } r \text{ in } t) : X}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C s : JA \quad \Gamma; \Delta, a : A, a' : A; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } s \text{ in } (\text{let } J(a') \text{ be } s \text{ in } t) = \text{let } J(a) \text{ be } s \text{ in } (t[a/a']) : X}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C s : KA \quad \Gamma, a : A, a' : A; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } s \text{ in } (\text{let } K(a') \text{ be } s \text{ in } t) = \text{let } K(a) \text{ be } s \text{ in } (t[a/a']) : X}$$

The equations for the interaction between $\mathcal{A}$ and C are:

$$\frac{\Delta \vdash_{\mathcal{A}} u : A \quad \Gamma; \Delta, a : A; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } J(u) \text{ in } t = t[u/a] : X}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C t : JA}{\Gamma; \Delta; \Phi \vdash_C \text{let } J(a) \text{ be } t \text{ in } J(a) = t : JA}$$

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A \quad \Gamma, a : A; \Delta; \Phi \vdash_C t : X}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } K(u) \text{ in } t = t[u/a] : X}$$

$$\frac{\Gamma; \Delta; \Phi \vdash_C t : KA}{\Gamma; \Delta; \Phi \vdash_C \text{let } K(a) \text{ be } t \text{ in } K(a) = t : KA}$$

$$\frac{\Gamma; a : A; \emptyset \vdash_C t : X \quad \Gamma \vdash_{\mathcal{A}} u : A}{\Gamma; \emptyset; \emptyset \vdash_C (\lambda(a : A).t) \bullet u = t[u/a] : X}$$

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A \Rightarrow X}{\Gamma \vdash_{\mathcal{A}} \lambda(a : A).u \bullet a = u : A \Rightarrow X}$$

$$\frac{\Gamma, a : A \vdash_{\mathcal{A}} u : B \quad \Gamma \vdash_{\mathcal{A}} v : A}{\Gamma \vdash_{\mathcal{A}} (\lambda(a : A).J(u)) v = u[v/a] : B}$$

$$\frac{\Gamma \vdash_{\mathcal{A}} u : A \Rightarrow JB}{\Gamma \vdash_{\mathcal{A}} \lambda(a : A).J(u a) = u : A \Rightarrow JB}$$

Received 2025-07-10; accepted 2025-11-06